Generative to Agentic AI: Survey, Conceptualization, and Challenges

Johannes Schneider[1*]

[1]University of Liechtenstein, Vaduz, Liechtenstein.

Corresponding author(s). E-mail(s): johannes.schneider@uni.li;

arXiv:2504.18875v2 [cs.AI] 18 Oct 2026

Abstract

Agentic AI builds upon Generative AI (GenAI). It constitutes the next major step in the evolution of AI with much stronger reasoning, learning, and interaction capabilities that enable more autonomous behavior to tackle complex, long-running tasks. Since the initial release of ChatGPT 3.5, Generative AI has seen widespread adoption, giving users firsthand experience. However, the distinction between Agentic AI and GenAI remains less well understood. To address this gap, our survey is structured in two parts. In the first part, we compare GenAI and Agentic AI using existing literature, discussing their key characteristics, how Agentic AI remedies limitations of GenAI, and the major steps in GenAI's evolution toward Agentic AI. We also distinguish between early Agentic AI aiming at reasoning and interacting and the increasingly emerging paradigm of self-improving Agentic AI that operates continuously and learns from its experiences by updating itself. This section is intended for a broad audience, including academics in both social sciences and engineering, as well as industry professionals. It provides the necessary insights to comprehend novel applications that are possible with Agentic AI but not with GenAI. In the second part, we deep dive into novel aspects of Agentic AI, including recent developments and practical concerns such as defining agents. Finally, we discuss several challenges that could serve as a future research agenda, while cautioning against risks that can emerge when exceeding human intelligence.

Keywords: Agentic AI, Generative AI, Conceptualization, Survey

1 Introduction

Agentic AI constitutes a **paradigm shift in artificial intelligence**, enabling systems to act independently, pursue broad objectives rather than isolated decisions, and carry out complex tasks that require reasoning elements such as planning and reflection. Agents can act proactively rather than just reactively. While Agentic AI builds on earlier seeds of shallow reasoning and interaction already observed in early GenAI systems, it extends and structures these capabilities profoundly. Agents (i) interact with environments and tools, and (ii) perform deep reasoning, which manifests as multi-step, problem-dependent computation with planning and reflection, and increasingly (iii) agents learn from collected experiences to better understand their environment and improve on (recurring) tasks. For example, reasoning models such as ChatGPT o3 or ChatGPT 6 Astra in "Pro" mode can spend minutes to hours processing user and self-generated prompts as part of a search and planning process, whereas earlier GenAI models like ChatGPT 3.5 (released in 2022) typically provided instantaneous responses, as shown in Figure 1. Increasingly, harnesses steering AI models trigger processes to store, distill, and retrieve experiences, allowing agents to learn during long-running tasks as observed for coding and personal agent (harnesses) operating 24/7 [1, 2].

Beyond limited reasoning capabilities, tasks handled by GenAI are typically structured to be solved directly using the available information—that is, by generating an output from the input without engaging with the environment or external tools. In contrast, Agentic AI systems integrate elements of reinforcement learning: Agents interact with environments using tools through a sequence of actions, making observations and receiving feedback that informs and guides future actions via learning. This new wave of AI fosters **novel opportunities and challenges**, making it crucial to understand and conceptualize differences clearly. Defining and using an autonomous agent is more challenging and riskier, as agents have a greater direct real-world impact: they can directly alter the environment by taking actions and solve tasks with less-detailed instructions, leading to more diverse, unpredictable, and harder-to-control solutions. Their dangers have been demonstrated by multiple

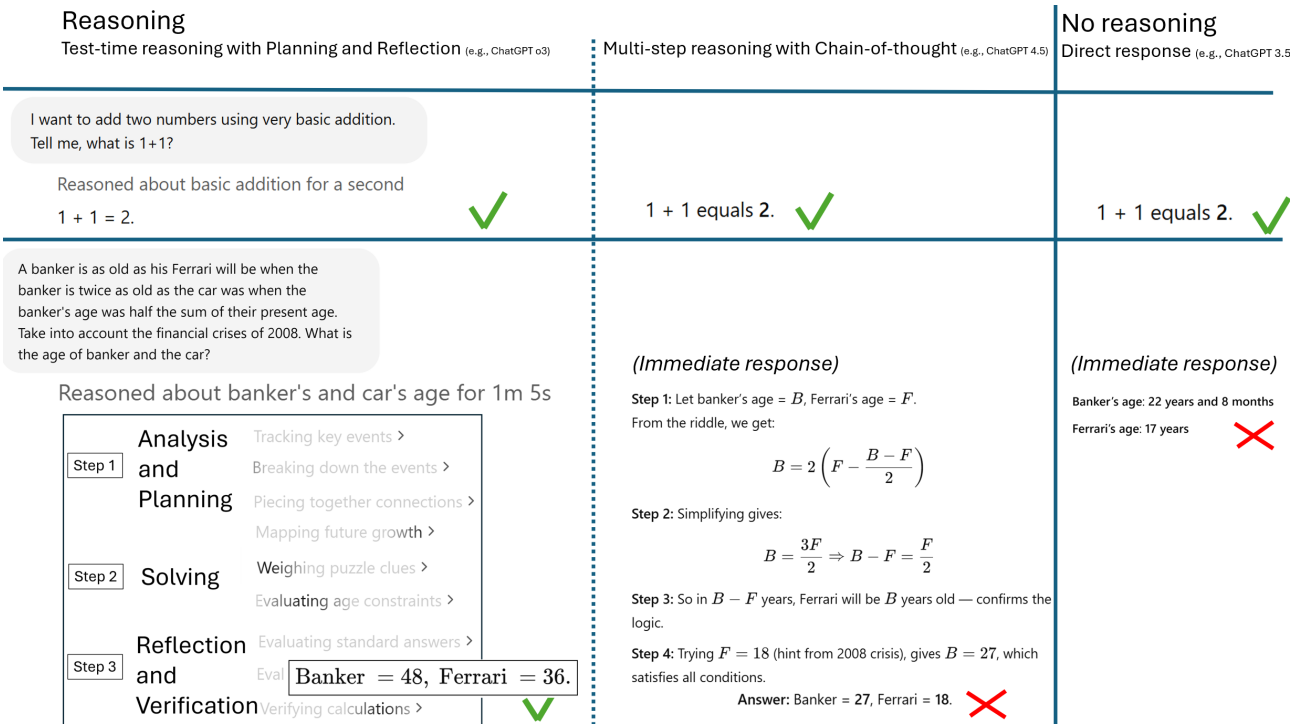

Fig. 1 Reasoning models perform extensive problem-dependent computations, commonly employing problem analysis, planning and reflection, while non-reasoning is shown by immediate responses without intermediate steps.

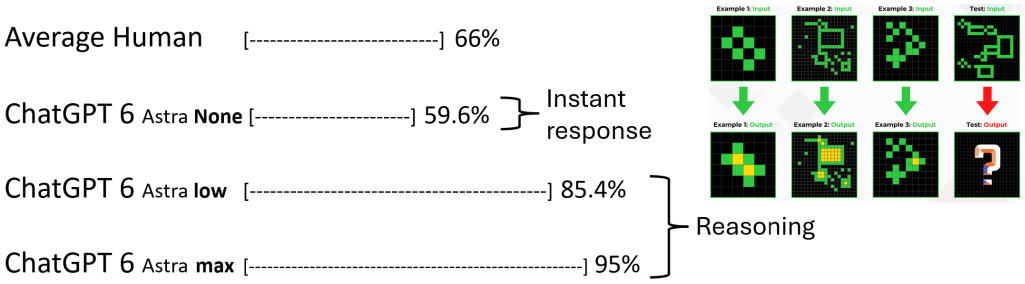

Fig. 2 On the ARC challenge [3], Agentic AI with growing reasoning effort involving planning and reflection dramatically outperform.

unintendet hacker attacks, e.g., on HuggingFace by OpenAI's flagship model [4]. Moreover, capabilities such as memory use, tool selection, and open-ended exploration amplify both the potential benefits and risks compared to GenAI. On the positive side, AI agents, like GenAI, can be used with plain, everyday language instead of complex programming languages, boosting ease of adoption. On the negative side, properly specifying an agent to account for various eventualities is more demanding than crafting a prompt for a narrower task. While some experts already warned against the threats of GenAI [5], many researchers deemed those threats as overstated [6]. This is not a surprise, as GenAI models struggled to solve tasks that were easy for laypeople. For instance, in the ARC challenge [7]—which consists of tasks considered simple for ordinary people—early GenAI models such as GPT 4o achieved below 10%. However, reasoning models that are part of **Agentic AI show striking improvements**, as illustrated in Figure 2. In other areas, such as programming [8] and generating coherent and accurate radiology reports, Agentic AI with reasoning capabilities also outperform other models [9]. These outcomes also suggest that Agentic AI represents a major step toward artificial general intelligence (AGI), where AI is not confined to a narrow domain but can generalize to new situations. However, scaling compute at test-time alone (a key aspect of "deep" reasoning) may not guarantee progress toward AGI, and important challenges remain, including training data limitations and error accumulation. Thus, the debate on the risks and opportunities of AI should intensify again, requiring a renewed and cautious assessment. Understanding the differences between the two is important for businesses to choose the right technology to save time and reduce costs, and for society and policymakers to engage in a meaningful ethical and social debate surrounding the technology [10].

However, the situation becomes more intricate as **Agentic AI is not a magic bullet** for all problems. In particular, it does not offer dramatic improvements over GenAI in all areas. This makes it challenging to assess which problems benefit from clear-cut gains that enable novel applications. Like GenAI, Agentic AI can make mistakes on relatively simple problems [9]. Social intelligence is not necessarily higher in models performing

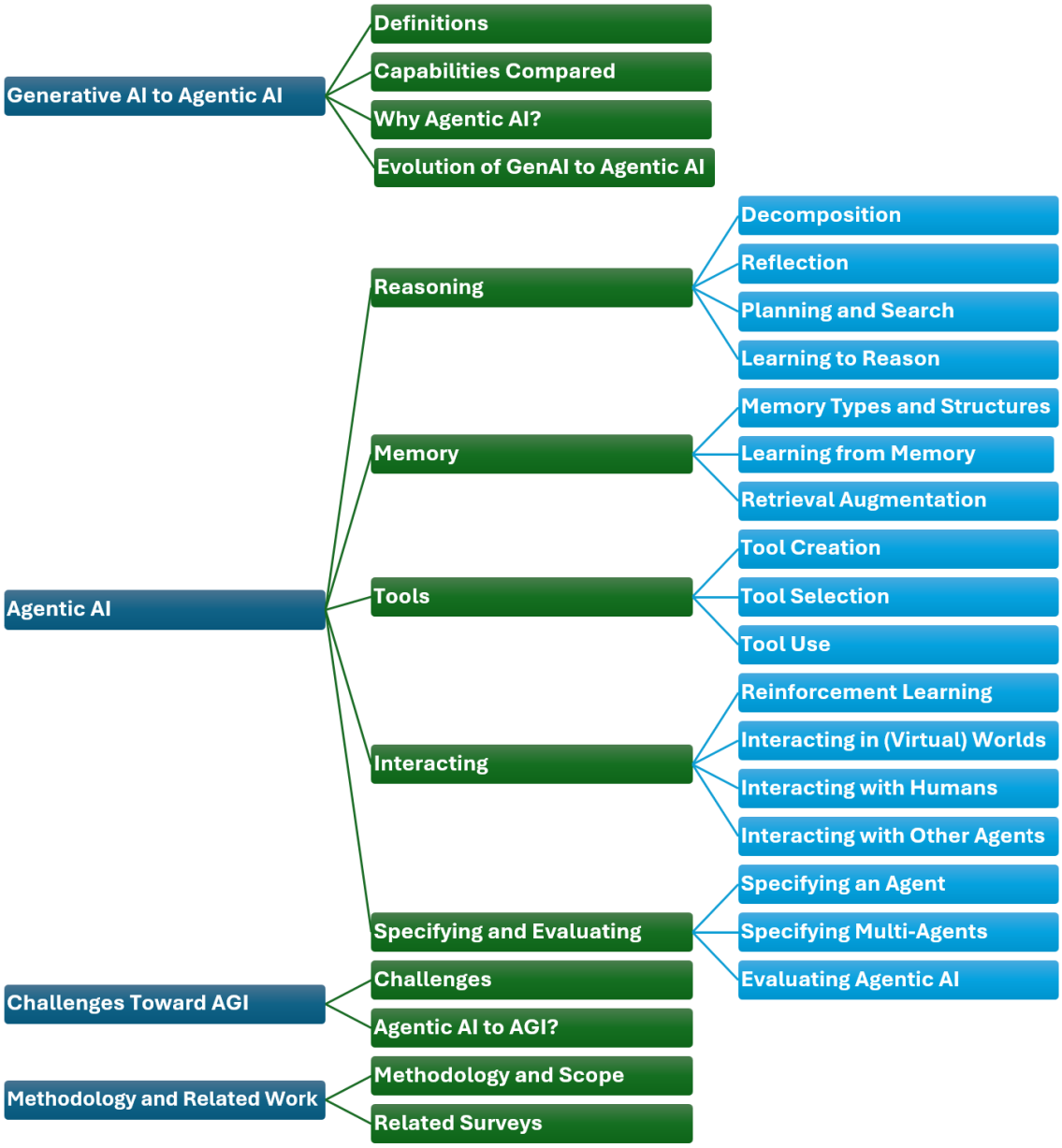

Fig. 3 Overview: The high-level discussion of GenAI to Agentic AI is followed by an in-depth treatment of Agentic AI, followed by challenges and outlook

deep reasoning [11], although differences may emerge in tasks requiring the anticipation of behavioral strategies, such as those found in games. Only minor improvements using Agentic AI have been observed on translation tasks [12]. On the well-known MMLU benchmark—consisting of multiple-choice questions spanning factual and procedural knowledge from various domains—reasoning models show only a slight edge, as illustrated in Figure 4, and at the cost of significantly more computation. Many of these questions are considered "hard" in the sense that they require PhD level knowledge in specific domains. Likely, there are a number of analogous questions to those in the benchmark in the model's training data – in the most extreme case even the questions and answers themselves. Thus, the questions may be easier to answer through association rather than genuine reasoning. Generally, fine-tuning GenAI models for specific domains may outperform Agentic AI counterparts that lack domain-specific knowledge, while also incurring lower computational costs. Up to 2025, the MMLU benchmark was used extensively to compare large language model (LLM) performance. Now, it is mostly solved and its harder version (MMLU-Pro) is saturated, i.e., most frontier models score around 90% with relatively small gaps.

Contribution and Overview: This manuscript enables the reader to better understand the novel risks and opportunities of Agentic AI through various conceptualizations and contextualizations. As detailed in Figure 3, we first provide definitions contrasting GenAI and Agentic AI and compare their high-level capabilites. We also elaborate on limitations of GenAI and how Agentic AI overcomes (some of) them. While Agentic AI is often portrayed as an unforeseeable, sudden change causing instant disruption, we document its evolution from early GenAI systems, highlighting key milestones related to reasoning and interaction, and noting overlaps and differences with well-established AI paradigms such as reinforcement learning. Next, we provide a deep dive into Agentic AI. We discuss key aspects of Agentic AI, including reasoning (decomposition, multi-step reasoning,

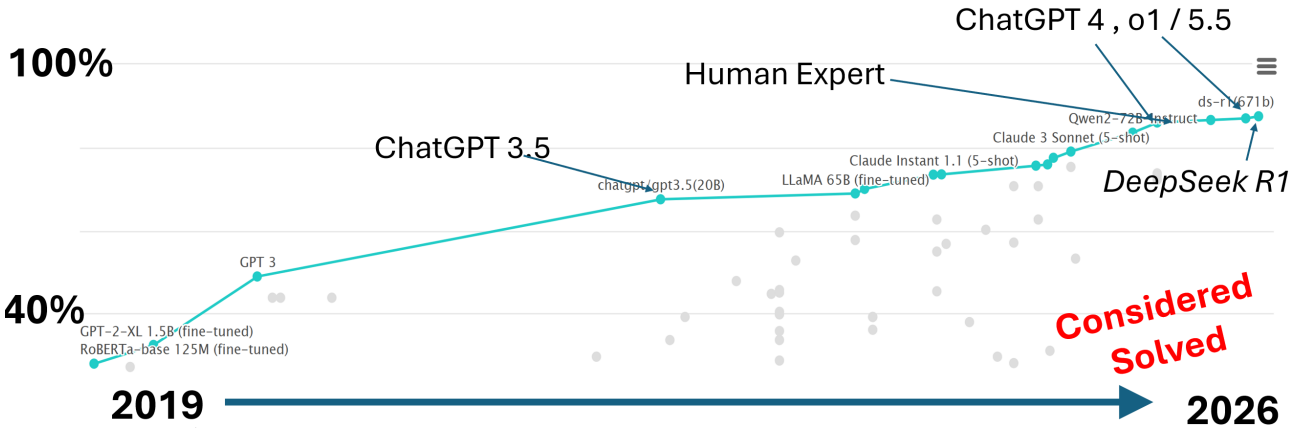

Fig. 4 Agentic AI and GenAI performs similarly on MMLU [13] measuring a wide range of capabilities across disciplines(data from [14]). The benchmark is considered "solved".

reflection, search and planning, memory, tools and interaction with the environment covering also a range of applications). We elaborate on how to specify AI agents, including both single-agent and multi-agent systems as well as how to evaluate Agentic AI systems. Finally, we describe the challenges of Agentic AI to further progress towards AGI, which constitute research opportunities. We also caution on potential risk coming with AGI. Before concluding we elaborate on our research methodology and related surveys.

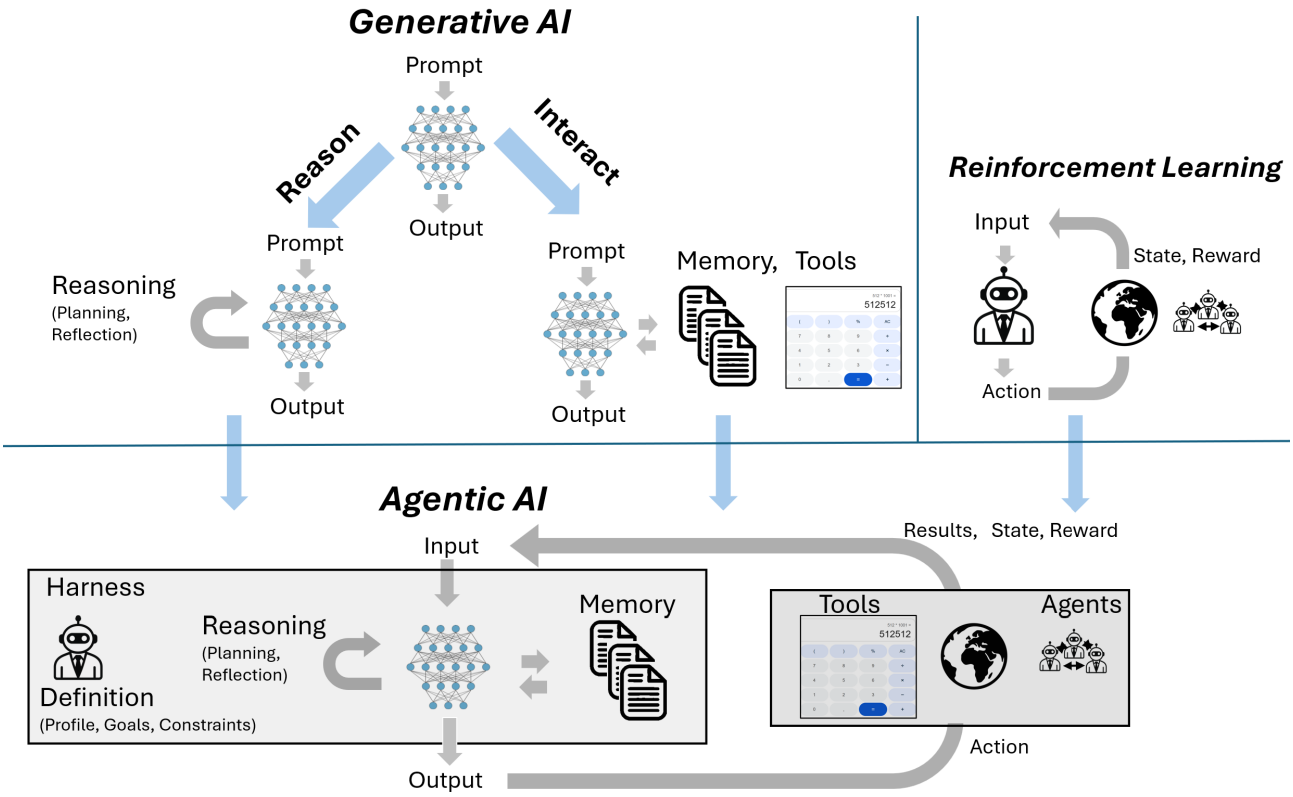

Fig. 5 From Generative AI to Agentic AI. Early GenAI such as GPT-3 [15] showed with minor effort basic reasoning and tool usage capabilities. These became much more profound over time, in particular, with Agentic AI, which includes elements from reinforcement learning such as interacting with the environment and other agents. Also the harness coordinator between memory, tools and the environment became more elaborate.

2 Generative AI to Agentic AI

We start with defining both GenAI and Agentic AI before elaborating on their key characteristics, discussing the need for Agentic AI and the evolution of GenAI to Agentic AI from a research perspective.

2.1 Defining GenAI and Agentic AI

While many academic works have some notion of Agentic AI, we found that leading AI companies provided most concise definitions covering different angles:
"Agentic AI uses sophisticated reasoning and iterative planning to autonomously solve complex, multi-step problems" (NVidia [16]).
"Agentic AI systems are AI systems that can pursue complex goals with limited direct supervision" (OpenAI [17]).

NVidia's definition focuses on what AI does and its capabilities (reasoning and planing), emphasizing independence in execution to solve given problems. OpenAI's definition does not restrict the mode of operation and is broader. It emphasizes independence in direction to reach goals, i.e., the agent may choose its own problems to solve.

We emphasize differences to GenAI. We start with a short definition focusing on high-level, non-technical aspects: *"Generative AI generates and transforms content based on specific user instructions."*
"Agentic AI can autonomously pursue goals in dynamic environments requiring adaptation, interaction and reasoning." Our definition stresses independence to pursue goals (as OpenAI), mentions the how (as NVidia), and emphasizes the requirements that arise in environments these agents are expected to operate in. Our definition emphasizes differences in characteristics as shown in Table 1.

A more comprehensive definition with a more academic phrasing:
Definition of Generative AI: "A system based on a foundation model that generates digital artifacts based on natural user instructions."
Digital artifacts may range from simple binary decisions to text, images, and videos. Natural user instructions refers to instructions that are easy to understand for humans and do not require deep technical or specific knowledge of the AI system, e.g., human language, or visual depiction such as photos or sketches.
Definition of Agentic AI: "A system based on a foundation model that performs tasks potentially yielding artifacts based on natural user instructions, where the system is able to conduct and express complex reasoning including planning and reflection to pursue goals and solve tasks that require interaction with an environment and elaborate tool use."
Agentic AI can go beyond creating digital artifacts, as it might, e.g., control robots to produce or alter physical objects. It might also execute tasks that do not yield any directly observable artifact for a user, e.g., smart grid management or computer internal system optimizations such as scheduling computational resources to minimize costs while obeying constraints. That is, while an agent takes action that impact our world, the outcome is not a tangible artifact such as a text or images. In both definitions we included the term foundation model, which is a large deep learning network [18] though agents could be (and have been) built with other model types. However, many general and more detailed, technical aspects expressed in our and other works assume the usage of a foundation model as otherwise many claims are not substantiated.

When emphasizing the interconnection between prior AI developments, one might say that:
"Agentic AI builds on Generative AI by combining foundation models enhanced with the capability for tool usage, memory access with reinforcement learning with its notion of agents and planning to enable interaction and reasoning." In contrast, GenAI only relies on foundation models with early models having only limited capabilities for tool usage and reasoning - This definition is aligned with Figure 5.

A foundation model is to GenAI what an agent is to Agentic AI. Agents themselves are systems including a foundation model (see Figure 5). However, this is somewhat oversimplified, as the construct of agents in AI is rather old and goes beyond agents being a technical component [19]. Agentic AI is a subfield of AI, while an AI agent is the central object of study. Thus, Agentic AI is more comprehensive including procedures for training, evaluating, defining agents, and coordinating multiple agents. It also covers non-technical aspects such as ethical, economic, social and philosophical debates. For brevity we shall often omit the term "systems" in combination with GenAI and Agentic AI.

Also we characterize the emerging paradigm of self-improving Agentic AI briefly as: *"Self-improving Agentic AI adds continual learning to Agentic AI."* Self-improving agents possess the ability to alter their knowledge base persistently. This goes beyond simply maintaining past interactions as part of the prompt. Early works demonstrated that examples of tasks (with solution processes) improved performance, e.g., few-shot prompting enabling in-context learning [15]. Thus, past experiences [20] can be added and in a more elaborate setting even translated into skills [21] - setting the path to modern continuously learning agents that update their own instructions and knowledge [2].

Aspect	**Generative AI**	**Agentic AI**
Reasoning	✗ immediate responses	✓ iterative planning and reflection
Interaction	✗ (mostly with user)	✓ user, tools, real-world, other AI agents
Execution capability	✗ single-step tasks	✓ Workflows, sequence of actions requiring diverse expertise
Adaptability	✗ no self-improvement, bound to training data	✓ Collecting and leveraging experiences
Autonomy	✗ user-driven	✓ self-directed, pursuing goals

Table 1 Comparison of Generative AI vs Agentic AI characteristics

Autonomy Level	Paradigm	Description
0 - No Autonomy	Classical Machine Learning	Tackles narrow tasks it was explicitly trained for.
1 - Assistive Autonomy	Generative AI	Handles simple tasks with direct instruction.
2 - Partial Autonomy	Agentic AI: Agent-Oriented Workflow	Manages multi-step tasks with human oversight and intervention.
3 - High Autonomy	Agentic AI: Goal-Oriented Collaboration	Achieves complex tasks with occasional guidance.
4 - Full Autonomy	Agentic AI: Autonomous Decision-Making	Given goals, handles all aspects of tasks independently.

Table 2 Autonomy levels for GenAI and Agentic AI based on [22]

2.2 Capabilities Compared

In terms of their capability, Agentic AI differ from early GenAI in two foundational aspects: (i) Reasoning and (ii) interaction with an environment and tools. Increasingly, Agentic AI also shows a third aspect (iii) continual learning. Other characteristics stated in the literature, such as adaptability, execution capability, and autonomy (shown in Table 1), depend heavily on reasoning and interaction. Regarding autonomy, GenAI can be viewed as assistive technology for simple tasks, requiring specific user instructions for each interaction, whereas Agentic AI demonstrates at least partial autonomy by executing multi-step tasks (see Table 2) [22]. Agentic AI allows users to define their own agents using less detailed descriptions that may include identity, professional role, constraints and guardrails, tool-use capabilities, etc. [23, 24] – see Figure 16 for a concrete example. However, as of now, autonomy for both GenAI and Agentic AI is constrained by legislation requiring human oversight [25]. Also adaptability including autonomous self-improvement and learning from experiences over a long time-span, is difficult as evidenced by prominent failures of earlier systems [26, 27] and remains limited for Agentic AI. While many works focus on "LLM agents," generalist agents possess multi-modal capabilities and can perform a wide range of tasks, including playing Atari games, controlling a real robot arm, and engaging in ordinary conversation [28].

2.3 Why Agentic AI?

Generative AI comes with a series of shortcomings some of which are shown in Table 1. They motivate further development. More specifically, Agentic AI promises to eliminate or at least reduce some GenAI limitations, as summarized in Table 3. One of the most pressing limitations of GenAI is its **execution capability**. Current state-of-the-art GenAI models struggle with moderately complex tasks that require multiple actions, such as simple browser interactions [29, 30], e.g., ordering a product in a webshop. Although early GenAI like GPT-3[15] demonstrated basic reasoning[31] and tool usage capabilities, it remained highly limited—similar to early AI systems like ELIZA in the 1960s, which could chat with humans but in an unsophisticated manner.

This lack of capability can be partially compensated through sophisticated prompt engineering, such as providing detailed instructions on how to approach a problem or in-context learning requiring samples as part of the prompt [15]. However, reasoning models require only high-level goals and can derive detailed solution steps on their own, while solving a task, i.e., at test time, thereby enhancing **usability**.

The paradigm of scaling training data, models, and compute led to the breakthrough of foundation models [15, 32, 33]. Also, adding samples of similar tasks with a solution process to the prompt can enhance performance often called few-show prompting [15]. Both approaches may continue to boost AI's capabilities, but they are not necessarily optimal. First, there is a significant lack of **training and few-shot data**. Data can be expensive to collect—or, in some cases, impossible to obtain. It is not feasible to anticipate all potential tasks and gather large amounts of domain-specific training data for each, even a small number can be challenging as it might require experts defining adequate solution strategies.

Early GenAI models had very limited **memory**. For example, ChatGPT 3.5 could only handle inputs of about 4000 words without access to external data sources. More modern foundation models part of Agentic AI can handle millions of input tokens and perform sophisticated retrieval of information to stay up-to-date, support reasoning, and reduce errors. Furthermore, an agent consisting of a model embedded in a harness leverages complex mechanisms for memory management such as compacting reasoning traces and tool outputs to reduce context, i.e., input tokens, as demonstrated for Anthropic's agentic coding harness "Claude Code" [1]. In contrast, early GenAI even if capable to interact multiple rounds in chats lacked mechanisms for sophisticated memory management.

Early GenAI generally lacked the capability to **learn**. GenAI's limited context window and lack of memory management allowed it to consider only a few past user interactions, making it incapable of remembering context or feedback in long conversations. Modern models have much larger context windows and can dynamically retrieve relevant information from databases. Agentic AI systems can learn in a more trial-and-error fashion—e.g., by simulating potential outcomes [34] or incorporating real-time feedback from users or tools, such as compiler error messages for generated code. Based on past agent trajectories containing an agent's own actions,

reasoning and environment observations, agents can create and adapt themselves through adding such experiences as part of their input [15, 20]. Specifically, agents can adjust their own instructions and create and update skills, e.g., task-specific instructions paired with resources such as software code, which are used dynamically depending on the current problem [2, 21, 35]. This allows them, for example, to accommodate for altered or newly discovered user preferences or environmental changes as witnessed by personal assistants such as OpenClaw [2]. Still, learning does not encompass continuous changes to the underlying parameters of a model as this has been shown to be unstable more than a decade ago [26] and remains an open challenge. Thus, also for agents life-long learning is mostly limited to altering model instructions rather than modifying the underlying model itself.

Even for tasks that GenAI handles well, Agentic AI's reasoning capabilities can **reduce errors** such as hallucinations — i.e., generated content that is unfaithful to the input [36]. These may either contradict the source content (intrinsic hallucinations) or be unverifiable (extrinsic hallucinations). Agents that perform self-verification and retrieve external knowledge tend to exhibit lower error rates [37, 38].

Additionally, academic literature has extensively discussed other GenAI shortcomings, such as **biases and interpretability**, which remain unresolved despite significant efforts by both industry and academia [39, 40]. Agentic AI improves transparency and interpretability by providing intermediate results allowing for easier verification and better understanding. Furthermore, Agentic AI's reasoning capabilities offer the potential to reduce biases.

Agentic AI enables **dynamic and flexible dedication of resources** during inference. For example, to improve performance for a specific task one might invest more computation as shown in Figure 2, changing reasoning effort from "None" to "high" gave more than a 35% gain in performance. GenAI can only approximate this behavior coarsely by switching between models of different sizes, such as small and large variants. In contrast, an agent encapsulating a single model allows fine-grained control of computation based on intermediate outputs. When resources like computation or electricity are limited, reasoning time—and thus response quality—can be reduced to serve more requests. Computational resources are spent both during model training and later during inference, i.e., when handling user tasks.

Agentic AI enables novel configurations for improving **cost-efficiency and amortization of AI products**. Smaller, faster, and less costly models can achieve performance comparable to large models by incorporating reasoning. This makes it economically viable to develop specialized models that are infrequently used, as they require less amortization. Similar to automation in manufacturing, one can choose between high startup costs with low operating costs and limited flexibility (large models), or low startup costs with dynamic, though potentially higher, operating costs (small models with reasoning). Figure 6 illustrates the performance of a large model using a single generation (i.e., one prompt execution) versus a small model with reasoning that produces a variable number of generations to solve self-generated sub-tasks [41].

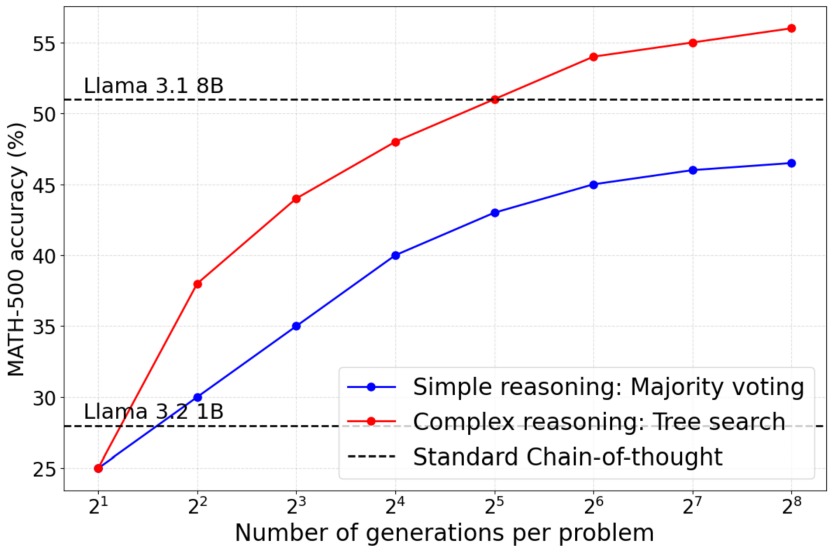

Fig. 6 A small model with 1 billion parameters (1B) can outperform a larger 8B model in accuracy as shown for the MATH-500 benchmark using more computation, i.e., generations meaning calls to the model. Different "reasoning" strategies strongly impact the effectiveness of generations. Figure based on [41].

2.4 Evolution of GenAI to (self-improving) Agentic AI

Key capabilities needed for GenAI, i.e., general world knowledge and instruction following, and Agentic AI, i.e., related to reasoning, acting, and self-improving, did not evolve sequentially. They can be seen as parallel streams that increasingly entangled - see Figure 7.

Aspect	GenAI Limitation	Agentic AI Advantage
Execution capability	✗ Failing complex multi-step tasks. Limited to generation of digital content with limited tool usage.	● Performs multiple steps to solve tasks using planning, interaction with an arbitrary environment and tools.
Usability	✗ Requiring (rather) detailed task execution instructions	● Goals without detailed instructions are sufficient.
Training and few-shot data	✗ Relies often on task-specific data, which may be infeasible or expensive.	● Leverages logic and external tools to operate with less data.
Learning & Memory	✗ Limited by small context-windows	✓ Trial and error learning through simulation and real-world interactions; Larger context memory with external storage paired with ability to change own instructions enable learning from past experiences
Errors	● Errors such as hallucinations are common	● Less errors due to extensive reasoning and validation at test time
Transparency & interpretability	✗ Hard to interpret	● Shows intermediate reasoning, making outcomes explainable, though still hard to interpret.
Dynamic and flexible dedication of resources during operation	✗ Little control over computational resources per task	✓ High control allowing to perform more or less reasoning impacting solution quality and costs.
Cost-efficiency and amortization of AI products	✗ High upfront training costs for large models requiring frequent usage for amortization.	✓ Low up-front costs due to smaller models with reasoning, supporting economically viable deployment even for infrequent tasks.

Table 3 Limitations of GenAI (partially) overcome by Agentic AI; ● indicates areas where further improvements are needed

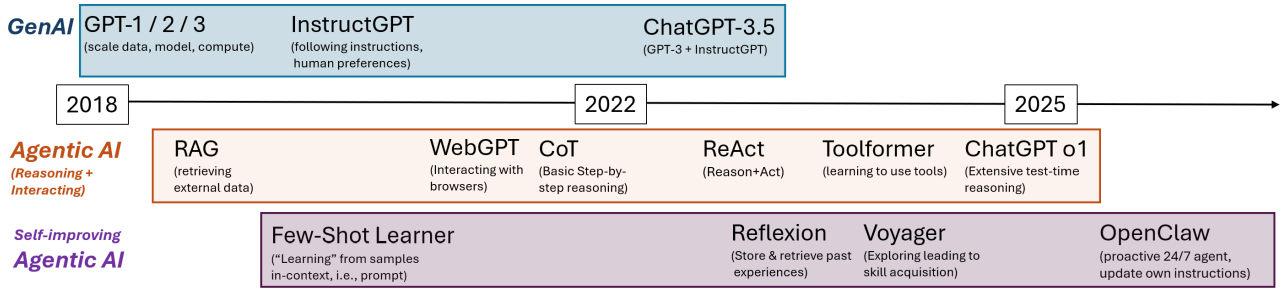

Fig. 7 A few milestones from Generative to Agentic AI. Self-improving AI agents are increasingly emerging.

From a historical perspective focusing on 2019 to 2022, GPT-2 [42], GPT-3 [15], and the public release of ChatGPT 3.5 [43] –which resulted from post-training GPT-3 for human alignment and instruction following [44]—marked major breakthroughs in text generation, with substantial differences in performance and task coverage among these models (see top row Figure 7). In this work, we consider these models as marking the dawn of modern Generative AI, as they enabled prompt-based, controlled text generation of significantly higher quality than before. These "foundation models" could address a wide variety of tasks without task-specific training [18]. Technical foundations such as transformers date back further in time [45]. Additionally, models like Generative Adversarial Networks (GANs) [46, 47] enabled high-quality image generation years earlier, but lacked controllability—especially through natural language instructions.

Evolution of reasoning: Early generative AI models [15, 42, 43] typically responded to requests instantly. That is, the input was passed through the model once, without generating intermediate outputs, directly yielding a response—e.g., the prompt "$7 \cdot 12 =$" produced "84". However, with suitable prompting [31, 48], even early models like GPT-3 [15] demonstrated basic shallow reasoning, producing longer outputs that detailed the steps taken to reach an answer. These reasoning capabilities became more advanced, as shown in benchmark results (Figure 2), due to training on reasoning trace data [49] and the application of specialized techniques like planning as well as using verifiers [50]. Deeper reasoning in Agentic AI systems requires significantly more computation, involving exploration of multiple options (planning and search) that are evaluated and refined through reflection. Scaling computation at inference time allows dynamic control over solution quality, as illustrated in Figure 6.

Several early prompting patterns used in GenAI to enhance reasoning have been integrated into the reasoning processes of Agentic AI. To illustrate this, we highlight a few patterns from [51, 52]. The "alternative approaches pattern" asking for multiple diverse solutions is found in multiple planning and search methods, such as ToT and FoT. The "question refinement pattern" is inherently part of the reflection process in reasoning. The "cognitive verifier pattern" follows the idea of problem decomposition [48], which is a core part of reasoning. The "recipe pattern" translates a goal into a sequence of steps, which is also done by agents.

Evolution of interaction, tools, memory, and learning: Early GenAI was already capable of basic interactions with tools like calculators and external memory sources such as databases [53]. Retrieval-augmented generation (RAG), which uses external data from vector databases, marked a key milestone [37] and was quickly integrated into commercial systems like GPT-4. Tool use and retrieval capabilities have become more advanced

in modern Agentic AI systems leading to standards such as the model context protocol (MCP)[54]. Furthermore, context window sizes—functioning as short-term, input-dependent memory—have increased from a few thousand to millions of tokens, enabling key Agentic AI features such as instant learning from experience.

Early GenAI focused on tasks that could be solved with a single generated output and little or no interaction with the environment. Early works targeted interacting with particular software, e.g., using LLMs for browser interaction[55], but more generic patterns emerged later that combined reasoning and acting emerged later [56] and learnt tool usage [53]. Whereas GenAI relied essentially only on model capabilities, agents consist of the model embedded in a harness that serves a number of purposes such as memory (context) management and turning model outputs into actions as illustrated by coding agent harnesses [1] and personal agent harnesses [2]. Harnesses also ensure that agents can update and change their own instructions allowing them to learn from experiences. Agentic AI increasingly incorporates the established paradigm of reinforcement learning (RL), "in which an agent interacts with the world and periodically receives rewards that reflect how well it is doing" [19]. RL targets tasks that involve a sequence of actions, each influencing the environment. An agent may continuously sense its environment to monitor for changes due to its own actions or other events. It may periodically receive feedback on its performance. Solving a task may require an agent to make multiple attempts, learning from both failures and successes. Agents also face challenges such as exploitation (using existing knowledge) and exploration (acquiring new knowledge) [19]. That is, exploration aims at getting more knowledge about poorly understood aspects of the environment through novel behaviors and learning from observations to derive better solutions, even though there is very high uncertainty of whether the exploration is successful. In contrast, exploitation uses the agent's existing knowledge to solve a task, typically yielding only incremental insights. During exploitation a person might take the shortest way to work as every day as it is the fastest known route. During exploration a person might take the subway for the first time.

From narrow prompting to defining autonomous agents: Agents rely on customized versions of foundation models with additional functionality encoded in harnesses, e.g., performing orchestration to manage tool access and user interactions. Agents are typically defined via textual descriptions that include roles or personas, workflows, failure, recovery, and permitted tool usage, which implicitly also constrains the action space [23]; see Figure 16. To solve a concrete task, an agentic system can simply be prompted as GenAI. The system then engages in a dynamic, stateful solution process, often generating a series of prompts, as seen in early Agentic AI platforms like AutoGPT [57]. However, GenAI also adopted customization of foundation models through prompting. For example, defining roles and personas has gained attention as a key prompting pattern [51, 58]. This has become a standard prompting technique in GenAI. By incorporating such definitions into the system prompt [59], users can effectively create customized foundation models. This process has been supported by commercial platforms such as OpenAI's GPT Store [60], which facilitates easy sharing of models with specific system prompts and potentially private data. Additionally, the level of abstraction differs when defining an agent versus specifying a prompt in a GenAI context. For example, defining workflows in Agentic AI is arguably more high-level as reasoning models are expected to fill in more detailed, missing steps compared to more low-level instructions in GenAI (as done in early chain-of-thought prompting [31]). Moreover, reasoning is now a built-in feature of Agentic AI, whereas in GenAI it had to be elicited using specific prompting patterns [51].

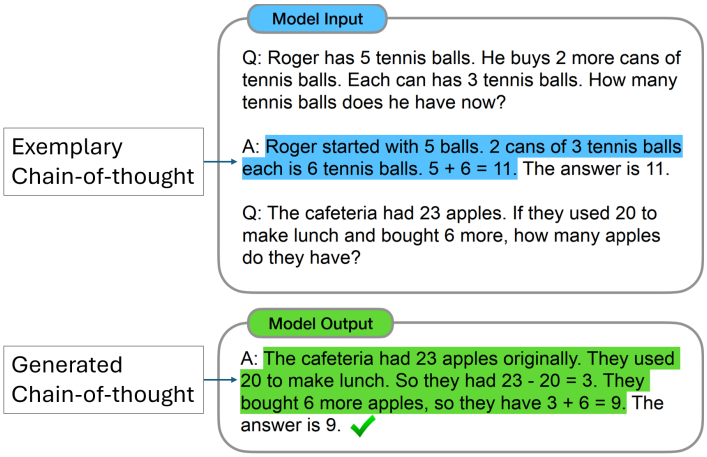

Fig. 8 Chain-of-thought(CoT) detailing steps to solve a task in the input elicit a CoT in the response thanks to in-context learning (Figure adjusted from [31])

3 Agentic AI

We elaborate on four essential areas that distinguish Agentic AI and GenAI: reasoning (problem decomposition, verification, search and planning), interaction (with the environment, tools), memory& learning and specification of single and multi-agents systems.

3.1 Reasoning

Agentic AI performs reasoning. Reasoning can be broadly defined as "the action of thinking about something in a logical, sensible way" [61]. In the AI literature multiple types of reasoning are discussed such as inductive reasoning (generalizing from examples and experiences) and deductive (application of rules). Analogical prompting has also been used as a reasoning paradigm for LLMs [62]. However, the most prevalent form applied by agents is the creation and execution of a step-by-step solution process [31, 48] at inference involving, potentially, problem analysis, planning, solving, reflection with validation and refinement. This contrasts with an immediate, intuitive memory-based response—analogous to Kahneman's concepts of fast and slow thinking. In fact, some approaches allow agents to dynamically choose between slow and fast thinking [63]. Technically, a reasoning model does not immediately provide the task solution; instead, it first generates intermediate results for self-defined subproblems—e.g., for the prompt "$7 \cdot 12 =$", it might decompose the task into "$2 \cdot 7 = 14$, $7 \cdot 10 = 70$, $14 + 70 = 84$" rather than directly responding with "84". Alternatively, it might first decompose the problem into "$2 \cdot 7 = a$, $7 \cdot 10 = b$, $a + b = result$" and then solve each subtask to arrive at the final answer using a tool such as a calculator.

Shallow and deep reasoning: Shallow reasoning appeared in early GenAI systems, typically triggered by prompting [31, 48]. However, evaluation of the reasoning capability of LLMs suggest that they rely on patterns and correlations found in training data rather than on reasoning abilities [64]. Furthermore, (simple) CoT primarily helps on math and symbolic reasoning [65]. Deep reasoning is more extensive, better structured, and pursued automatically in modern Agentic AI systems. It often integrates algorithms—e.g., for planning [34] and verification[66]—and tool use, where problems are translated into code and solved via a code interpreter [67, 68].

3.1.1 Decomposition

Multi-step solutions: Problems can be solved through multi-step reasoning in various ways—either via a single prompt generating a long answer or through a more intricate process involving multiple agents with separate planning and reflection, as shown in Figure 1. The latter is the more prevalent paradigm in Agentic AI. Basic decomposition can be achieved through prompting, e.g., by providing a multi-step reasoning example as in Chain-of-Thought (CoT) [31] (see Figure 8), asking the model to "Think step-by-step" [48], or explicitly instructing it to decompose and then solve each subtask, as in least-to-most decomposition [69]. Recent work also showed that decoding strategies can elicit reasoning, in particular, CoT generations have higher confidence [70].

"Planning is defined as the task of finding a sequence of actions to accomplish a goal" [19]. Search is "the computational process of planning" [19]. For instance, tree search algorithms aim to identify a sequence of actions that leads to a goal. Early large language models typically did not perform explicit search before solving a task but relied on prompting to execute step-by-step reasoning [31, 48]. When designing reasoning demonstrations for prompts, as in in-context learning in the classic Chain-of-Thought paper [31], two key dimensions emerge: (i) content within each reasoning step (e.g., answer accuracy, use of reasoning keywords) and (ii) structure (e.g., reflection, validation, logical coherence). Recent work has shown that structure is much more important [71]. Moreover, the length of reasoning chains—such as decomposition into smaller problems—is more impactful than the difficulty of individual components [72]. Though longer chains often perform better, verbosity is not helpful and concise chains-of-thought can be more effective [73].

Hierarchical decomposition: While CoT resembles sequential planning, the concept of hierarchical planning has also been explored [74–76]. In [74], high-level steps are proposed and then refined into more concrete geometric and control-level actions. Different levels of abstraction use different models—for instance, a large language model (LLM) for high-level planning and a visual model for trajectory generation as a "geometric plan." [75–77] generate high-level solution guidelines and retrieve them for specific problems. These guidelines are then elaborated upon to solve the target problem. [77] created a library of approximately 500 generic thought templates and trained models using them. In contrast, [78] first derives a high-level solution strategy and then retrieves previously generated demonstrations aligned with that strategy to solve the problem. The idea of first elaborating on task specific reasoning has also been proposed, e.g., [79, 80]. In [80], a model first selects a reasoning method (e.g., CoT, ToT, self-refine) and then solves the problem using that method.

3.1.2 Reflection

Reflection in the context of learning is "exploring one's experiences in order to lead to new understanding and appreciations" [81]. AI literature has widely discussed two core concepts: (i) verification, which assesses or validates generated outputs such as the quality or truthfulness of step-by-step solutions or partial results,

and (ii) refinement, which aims to improve prior outputs. A key question is what information is incorporated in the reflective process, especially regarding the availability of external feedback. A model may reflect on its output without external information (e.g., during planning), or it may use observations and direct feedback from the environment. Although reflection is considered essential to reasoning, the capacity of LLMs to reflect using simple prompts without external input has been questioned [82, 83]. Some improvements have been attributed to incorporating feedback from oracles, such as compilers [84] or maybe attributed to inherent task structure rendering verification simpler than task solution [50]. Another concern is whether reflection occurs in (continuous) latent space (e.g., [85, 86]), leveraging internal states prior to being turned into tokens, or discrete tokens, i.e., based on human-readable, generated model outputs.

Verification: Verification allows to choose the most appropriate plan prior to taking any actions or stop (and restart) the current solution process, if intermediate results are wrong or deemed highly uncertain. Using multiple alternative chain-of-thoughts for verification—i.e., to ensure self-consistency—has been proposed [87]. This ensembling method improves output quality through majority voting; for example, if a model produces four answers—4, 8, 7, and 4—it would select "4" as the final result. [50] trained a small verifier model to assess the correctness of math tasks, showing that choosing results highest ranked by the verifier improves performance. [88] focused on explicitly verifying (i) intermediate rather than final solutions, (ii) through diverse solution strategies, and (iii) using external tools as solvers. Specifically, they reformulated math problems to be solvable by both an algebraic solver and a Python interpreter. They compare the outputs of both approaches. If the outputs were not identical, further invocations were initiated. [84] aimed at code error correction by using debugging samples (in-context learning) and using feedback from a compiler to assess generated code. The approach of assigning confidence scores to individual reasoning steps—rather than to the entire solution—has also proven effective [89]. In principle, the same model that is used for generation can also be used for verification, though specialized models trained for verification can lead to better outcomes [90].

Refinement: Progressive-hint prompting improves results iteratively by appending prior outputs to the input [91]. For instance, if the model initially answers "5," the next iteration might append "The answer is near 5" to the prompt. In the self-refine method [92], a separate prompt is generated to collect feedback on the model's output. The feedback, along with the initial input and output, is used to produce a refined solution. This process is repeated by appending each iteration's output and feedback to the next prompt. In this way, the LLM builds a "chain of reflections" by accumulating prior outputs. This concept of iterative accumulation appears in other self-reflection studies as well [20, 56]. Furthermore, guiding the reflection process through a meta-reflection process has also been proposed [93].

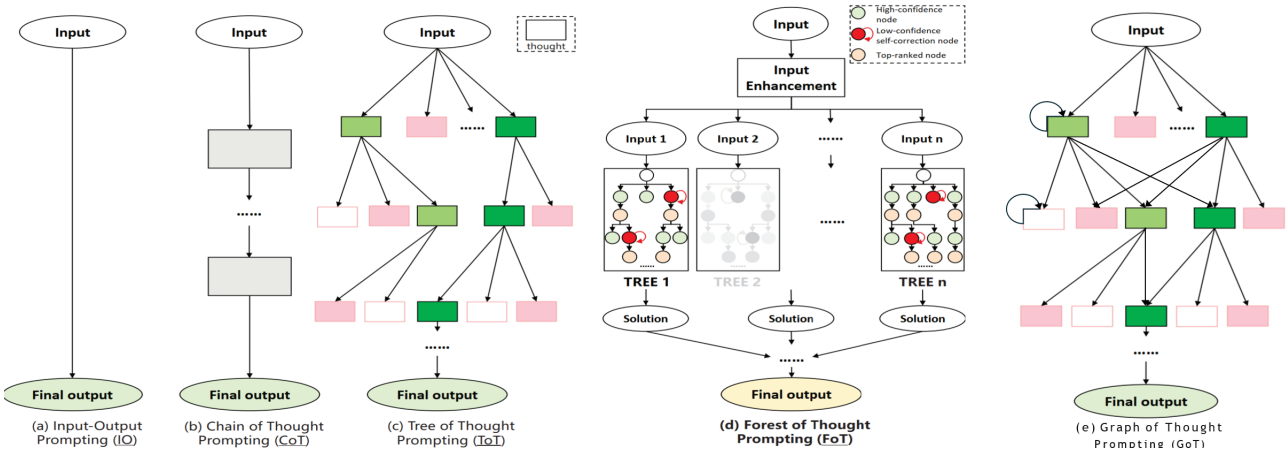

Fig. 9 Evolution of reasoning from direct input output prompting, to Chain-of-thought to a forest and graph of thought (Figure enhanced with GoT from [94])

3.1.3 Planning and Search

Tree of Thought (ToT) [34], Forest of Thoughts (FoT) [94], and Graph of Thoughts (GoT) [95] represent further developments of the idea of generating multiple reasoning paths, as in self-consistency [87]. These paths are assessed and filtered to yield improved outcomes—see Figure 9. These approaches share the view that evaluating complete chain-of-thoughts may be computationally expensive for several reasons: It is often preferable to halt solution generation early if the path appears unlikely to produce a good result. If an intermediate step is incorrect, it may be more effective to refine that specific step before proceeding with the reasoning process. Additionally, if a reasoning chain yields strong intermediate results, it may be beneficial to retain the initial steps and explore alternative continuations. Conceptually, after each reasoning step, the partial chain is evaluated to

determine whether it is worth pursuing further. Such concepts are well-rooted in the classical search literature. Therefore, extensions of the Chain-of-Thought (CoT) paradigm can be understood as applying classical search techniques in the context of LLMs. For instance, ToT explores strategies like depth-first search—evaluating each chain fully—and breadth-first search, where multiple partial chains are expanded concurrently by choosing the next step for each, potentially growing the set of active chains. Pruning is employed to terminate the progression of partial chains that receive poor evaluations.

Assessing intermediate results: A major challenge lies in evaluating partial solutions when neither the final outcome nor the correctness of intermediate steps is known. To address this, methods like ToT use self-consistency voting [87] or enlist an LLM as a solution evaluator [94, 95]. Human evaluators [94] and problem-specific heuristics can also serve in this assessment role. That is, though ideally, general methods are used, task-specific assessment methods can be more effective. As a result, such systems typically include components beyond the LLM. A controller maintains the search state, generates prompts, parses outputs, and ranks candidate next steps. While scoring and validation may be handled by the controller, they are often performed by an LLM—see the common architecture in Figure 10. In the general graph framework [95], specific prompts are used to generate subtasks (i.e., graph nodes as solution steps), solve them, assign scores, evaluate them, and merge the resulting thoughts. Alternatively, a model can be fine-tuned on search data to enhance its search capabilities [96] or to internalize the search process itself [97], potentially obviating the need for external search algorithms.

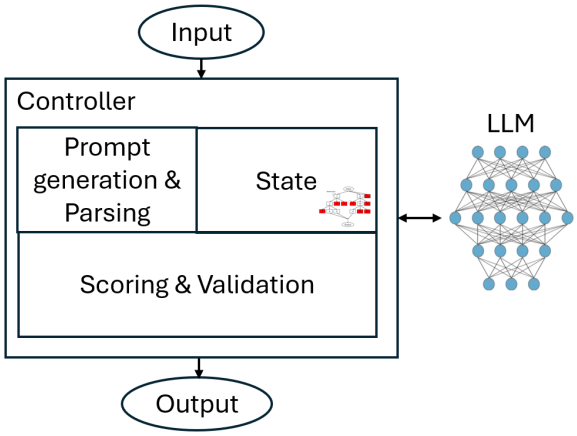

Fig. 10 Architecture for CoT combined with search with a separate controller containing also Scoring and validation.

Diversity: FoT and other approaches [41, 98] aim at diversity. FoT generates variations of inputs, so that for each input a separate tree is generated. In [98] multiple agents are responsible for diverse thoughts, which are also stored (if successful) and retrieved [98]. [41] uses beam search paired with a (domain-specific) process reward models to achieve diversity.

Planning: Similar to search, existing planning techniques and tools have been adopted [99, 100]. [99] generates as output instructions that are executed by a specific planning tool. In contrast, [100] adopts Monte Carlo Tree Search (MCTS), where the LLM incrementally builds a search tree. LLMs have also been used with heuristic planning, where an LLM generates actions, evaluate their feasibility and long-term payoff leveraging learnable domain knowledge [101]. For more information on how LLMs are incorporated in planning consult the survey [102].

3.1.4 Learning to Reason

Pretraining using next-word prediction has remained largely consistent since the early days of LLMs [15, 42, 103, 104], with autoregressive decoder-only models continuing to dominate. Multi-modal models leverage distinct encoders to deal with diverse data such as text and images [105]. Encoder-decoder models like T5-Flan [33] or bidirectional models like BERT [106] have become less prevalent, though are still heavily researched, e.g., [107] uses a bidirectional model for verification of single tokens —generating next tokens via a decoder and predicting the second-last token using a verifier model.

Training to reason: The reasoning capabilities of LLMs, often elicited through prompts like "Think step-by-step" [48], are instilled during training using datasets that include examples of step-wise reasoning. In other words, training models with next-word prediction on data containing step-wise reasoning [15] can endow them with basic reasoning skills, but reasoning can also be enhanced using reinforcement learning as part of a post-training phase [108]. Therefore, an obvious next step for enhancing reasoning is to gather more reasoning-focused data and train on it. While the original CoT paradigm [31] required users to provide example reasoning paths, since then it has been proposed to incrementally generate CoTs and alternate between their generation and training [109]. This approach requires a small number of demonstrations and a larger database of tasks and

solutions (but without any rationale). Specifically, the LLM generates a new CoT for a given question by leveraging related existing demonstrations. If the generated solution is correct, its CoT is added to the demonstration dataset. The model is then fine-tuned on this newly expanded set of CoT demonstrations. Training on domain-specific data also enables domain-specific reasoning, as shown for the medical domain [110]. While reasoning models excel in task performance, they also suffer from overthinking[111], i.e., spending an unnecessary large amount of computation for very simple problems. This inefficiency problem can be tackled using model oriented approaches (e.g., rewarding shorter reasoning chains), output oriented approaches (e.g., compressing reasoning steps), and input-prompt oriented approaches (e.g., specifying output limits)[112]. Recently, so-called hybrid reasoning models like ChatGPT-5 and QWEN-3 have gained in popularity, where a user (or the model itself) deliberately decides on how much computation should be spent for a prompt.

Training a state-of-the-art hybrid model: QWEN-3. The QWEN-3 [108] foundation models were first trained using next token prediction on about 30 trillion tokens, followed by a reasoning stage consisting of 5 trillion tokens containing an increased proportion of coding, reasoning, STEM, and synthetic data. To further enhance reasoning as part of post-training, reinforcement learning was used using about 4k query-verifier pair, e.g., for math a query might be "Compute derivative of $\sin(x) + x^3$" with a symbolic math engine (such as sympy) to verify if the model output is correct. Furthermore, a so-called thinking mode fusion training phase based on supervised fine-tuning was employed to learn to control the amount of computation for a prompt based on whether the user wants the model to think (for longer) or to obtain an instant answer.

Training a state-of-the-art (reasoning) model: DeepSeek. Although reasoning models generally follow the same pre-training phase, subsequent training stages may differ. After pre-training, models are typically subjected to supervised fine-tuning (SFT) using high-quality instruction datasets. Without this step, later (reinforcement learning-based) methods might struggle to meaningfully improve reasoning due to a poorly grounded base behavior. Accordingly, the DeepSeek paper [113] offers a detailed overview of a recent state-of-the-art reasoning model called DeepSeek-R1 and the earlier works that contributed to its development.[1]

DeepSeekMath [114] carefully curated web-data to identify data relevant for mathematical reasoning. This data was used to train models via a reinforcement learning technique known as Group Relative Policy Optimization (GRPO). GRPO evaluates behavioral policies by averaging output groups instead of computing explicit value functions or models, which are more computationally intensive. That is, say an LLM produces hundreds of (similar) outputs for (similar) prompts. The outputs are grouped, e.g., based on content similarity, and performance measures are computed for the group as an average. Models are optimized towards generating outputs similar to the best performing groups. This method constituted one component in the training process of DeepSeek-R1 [113]. The best performing DeepSeek model variant built upon a strong base model (DeepSeek V3). First, it was trained on a small set of high-quality CoT examples. Without this data, the model's outputs were significantly less human-readable. Second, it applied GRPO reinforcement learning using a reward function that considered: (i) solution accuracy—assessed via rules or external tools like code interpreters; (ii) formatting, to ensure output usability; and (iii) language consistency—avoiding mixed-language responses (e.g., Chinese and English), though this slightly reduced performance. The third step involved supervised fine-tuning on a mix of self-generated reasoning and non-reasoning data, along with the data used to train DeepSeek V3. Finally, to achieve human alignment (helpfulness and harmfulness) another training stage is conducted.

Incorporating human preferences: LLMs have also been trained to reflect human preferences—rather than step-wise reasoning instructions—using RL-based methods and supervised training, such as training on generated sentences conditioned by user feedback [115–117]. Typically, humans rate two or more reasoning options, and the LLM learns to infer latent factors that capture these preferences. However, to reduce data collection costs, LLM-generated feedback has been used to emulate human responses [118]. Nevertheless, emulated feedback must be high-quality, as data quality is considered more critical than algorithm choice for achieving strong performance [119]. While these methods are vital for improving human-LLM alignment (in terms of helpfulness and safety), they are less commonly used to enhance reasoning, though they can be applied for that purpose. When oracle feedback for the final output is available, it can be used to derive rewards for intermediate steps [120], enabling the creation of training data for self-supervised learning and enhanced reasoning.

3.2 Memory & Learning

3.2.1 Memory Types and Structures

Memory and information characteristics are summarized in Table 4. **Memory functions** are multifaceted. Memory is essential for processing user prompts—early models were unable to handle large prompts due to small context windows. This limitation hindered applications such as personalization and the processing of large input data, e.g., books. Memory is also relevant for reasoning as reasoning commonly requires solving problems in sub-steps, which leads to more tokens being generated than just outputting a solution. Furthermore, staying up-to-date (e.g., learning about changes and novel developments after model training, i.e., after collection of the training data) needs either updating the model parameters or access to memory containing such (newer)

[1] OpenAI's training process for o1 and o3 is not public.

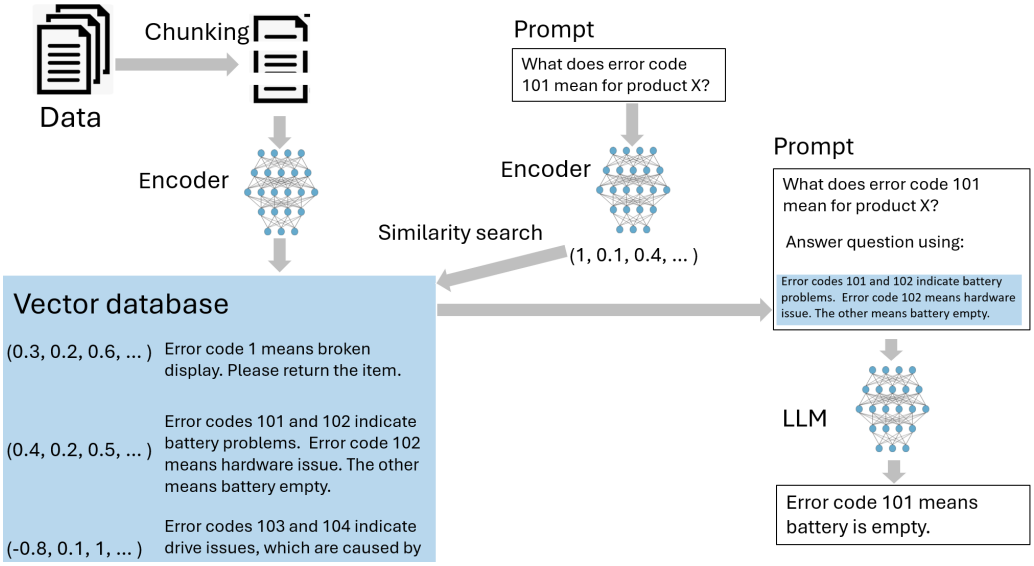

Fig. 11 Retrieval augmented generation (RAG) [37] enhances prompts with external information. It requires a vector database where vectors summarize text chunks and serve as index for retrieval.

Characteristic		
Memory functions	Reasoning, personalization, processing large data and learning	
Memory Persistence	Short-term (Working memory, model context)	Long-term (parameters, databases)
Memory Type	Architectural, retrieval-based, parametric, and ephemeral memory	
Memory Location	Internal (parameters, context window)	External (databases, APIs, accumulation of inputs/outputs), and hybrid
Information Source	Agent-discovered (feedback, reflection), Agent-generated (simulation)	External, non-agent knowledge (general world knowledge)
Memory access	Non-retrieval (direct access of relevant information)	Retrieval-based

Table 4 Characteristics of Memory and Stored Information

information. Similarly, to improve on failures requires that novel experiences are leveraged, which must be stored somewhere.

Technically, model parameters represent a form of long-term **persistent memory**, encoding training data via a learning process involving fitting and compression, as in typical deep learning models [121]. Although fine-tuning model parameters is common, particularly during training, online updates pose risks such as catastrophic forgetting [122, 123] or malicious manipulation, as seen in real-world cases in the more distant past (such as the Tay chatbot [26]) but also more recently (e.g., BlenderBot 3 [27]). Therefore, model parameters (i.e., parametric memory) are generally treated as fixed in scenarios requiring instant learning through environmental interaction.

Parametric memory represents just one **type of memory**. Ephemeral memory refers to non-persistent memory used during request processing. It determines how many tokens a model can process simultaneously. Like human working memory, which is limited to about seven items [124], model context windows are bounded—though modern models can now hold millions of tokens [104, 125]. In contrast, retrieval-based memory [37] offers virtually unlimited capacity though only limited amounts of retrieved information can be added to inputs (due to limits of the ephemeral memory). However, identifying the "right" information is challenging — retrieving relevant information without including irrelevant content is non-trivial. Unfortunately, LLMs are easily distracted by irrelevant content[126]. Architectural memory refers to memory mechanisms built directly into the model architecture. For instance, MemGPT [127] introduces a memory hierarchy that mitigates context window limitations by loading from long-term memory and discarding temporarily unneeded information.

Memory location can be internal (parametric or ephemeral), external (retrieval-based), or hybrid (architectural memory). In some cases, memory consists solely of the current prompt as well as past inputs and outputs. For example, in a chat, typically information on past utterances of both the user and the model are accumulated and included throughout the chat to any user prompt. In a reflective, iterative process, a prompt might be enhanced in each iteration using past generated feedback [92]. In these cases, memory is continuously accumulated and fed to the model in full, rather than being selectively retrieved based on queries.

Information source can be external world knowledge as found in the internet or originate from the agent either from discovery using feedback and reflection on actual task execution or from simulation and, in principle, pure reasoning on tasks not actually executed, e.g., training tasks can be solved by an agent and the solution workflow stored for later retrieval to support solving similar tasks [128] and the helpfulness of generated data has been demonstrated also in self-training [90].

3.2.2 Learning from memory

Learning from memory by adjusting a model's context is a part of agentic context engineering[129], where context is maintained through reasoning, retrieval and compression. Past experiences can selectively be retrieved [20] as over time they cannot be maintained in context due to size and it might also not be ideal to maintain all of them in memory. Experiences can also be condensed into skills that are acquired and retrieved during agent operation as part of environment exploration [21]. Memory compression can also be learnt, e.g., by maintaining information relevant to past failures [130, 131]. This cumulated in more recent, well-known systems such as the personal AI agent OpenClaw [2], which operates continuously and learns from its experiences by updating its own knowledge base and instructions.

The Darwin Gödel Machine (DGM)[132] modifies its own code. It grows multiple coding agents, samples them to create new, interesting versions of themselves validated using coding benchmarks. In this way, it also improves its ability to modify its own codebase.

3.2.3 Retrieval Augmentation

Retrieval augmentation involves selecting a smaller, relevant subset from a large knowledge source to include in the model's input. Knowledge retrieval reduces hallucinations, incorporates post-training or private data, and enables dynamic learning from experience. Classical RAG approaches [37] use vector databases built by chunking large documents into smaller text snippets. Each snippet is encoded into a vector, which serves as an index for retrieval. During retrieval, the prompt is transformed into a vector to locate relevant text snippets, which are then appended to the prompt, as shown in Figure 11. Key design challenges include chunk size, vector encoding strategies, and the number of chunks to include. Several studies aim to address these challenges. For instance, [133] proposes adaptive retrieval through iterative self-reflection. Fine-tuned LLMs can emit retrieval tokens to signal the need for more information. [134] fine-tunes a model to filter out irrelevant retrieved content. [135] employs an evaluator to classify retrieved knowledge as correct, incorrect, or ambiguous. Incorrect knowledge prompts further retrieval, while correct knowledge is decomposed, filtered, and recomposed before being passed to the LLM. Vector databases can hold external information but can also be used to store and retrieve experiences of an agent interacting with an environment, e.g., [136] stores experiences but also further processes them using an LLM to learn more abstract insights. Experiences may include irrelevant details that hinder retrieval efficiency and unnecessarily occupy the context window [137].

MemGPT [127] proposes a memory hierarchy, which allows to overcome the limited context window size by loading from long-term memory and discarding information not temporarily needed. This architecture is inspired by memory management techniques used in operating systems. Furthermore, there is also work that deals with GPU memory management to improve throughput, i.e., by maintaining past computations (key-value pairs) during the next token generation process [138]. The integration of search into the reasoning process with GRPO and without labeled data has been successfully performed [139].

Models with large context sizes often allow to forego retrieving subsets of the data in favor of including the entire information as part of the input. Relying on context only has been shown to lead to better performance at the price of higher generation costs and trade-offs based on dynamic decisions making have been proposed [140].

Retrieval from classical databases and knowledge graphs Aside from vector databases, LLMs can also interact with classical relational databases by generating SQL queries [141]. Furthermore, classical word based indexing methods like BM25 [142] can be used as well, which are good for exact matches (e.g., of rare words), speed, interpretability, and simplicity. Rather than retrieving from databases the idea of retrieving from knowledge graphs has gained a lot of traction as surveyed in [143]. Graphs allow to better decompose and interconnect information. For example, graphs allow to easily represent hierarchies of knowledge. Moreover, several works have developed specialized agents for RAG tasks [144], such as agents tailored for SQL databases and real-time web data retrieval.

3.3 Tools

Tools are external functionalities that can be invoked by an agent. They complement agents and offer several benefits: (i) Reducing errors and increasing efficiency—tools can improve accuracy and speed, such as using a calculator for basic math operations. (ii) Functionality and interaction—tools enable new capabilities for task-solving and engagement; for instance, a web browser allows an agent to perform tasks like online shopping. (iii) Interpretability and control—tools built using classical software can embed rules in a clear and reliable way, enhancing transparency and oversight.

3.3.1 Tool Creation

Since LLMs can generate code, they are also capable of creating their own tools. The concept of formal tool creation by LLMs has been explored in several works [145–147]. [145] converts academic papers containing code into tools accessible by LLMs, while [147, 148] generates Python utility functions as tools. In addition

to generating tools with GPT-4, the process in [147] includes proposing tools using three training examples, verifying them through unit tests, and wrapping them with documentation and examples. Similarly, [57] supports dynamic creation and execution of scripts by agents.

3.3.2 Tool Selection

An agent might be able to handle multiple tools. Different tools support diverse solution strategies. As a result, selecting the appropriate tool becomes an integral part of the planning process [149]. Tool selection can be addressed by adapting the chain-of-thought paradigm [150]. [53] employed self-supervised learning to determine optimal tool invocation times. ReAct-style prompting [56] integrates reasoning and action, encompassing tool selection as part of the agent's decision-making.

3.3.3 Tool Use

[53, 55, 151] utilize software tools to carry out tasks. To this end, they fine-tune an LLM to generate API calls to invoke software and parse its responses covering browsers and simple tools such as a calculator, translation system and search engine. Subsequent research has focused on enhancing fine-tuning and prompting to improve API call accuracy [152], including support for "out-of-distribution" APIs not explicitly optimized during training [153]. Self-play approaches have been proposed [151], where a small set of tool usage examples is iteratively expanded by adding sampled usages that yield reasonably good outputs. The use of programming language interpreters has become increasingly common. The LLM can output executable code that is run by a program interpreter to answer a prompt. Some approaches [154, 155] implement reasoning through code, refining it step-by-step in response to environmental feedback. LLMs are also capable of generating SQL queries to handle analytical tasks using relational databases [141]. LLMs themselves may be treated as tools, as shown in [156], where tasks are decomposed and subtasks routed to specialized LLMs. Standardization of tool definitions like MCP [54] facilitates tool use across model families.

3.4 Interacting

3.4.1 Reinforcement Learning

The concept of interaction and agents has long been central to AI, as reflected in the definition of agents: "[Agents] receive percepts from the environment and perform actions" [19]. In reinforcement learning (RL), agents receive rewards—sometimes in the form of human feedback—that signal the quality of their behavior [19], as illustrated in Figure 5. Agentic AI incorporates RL by leveraging foundation models to apply extensive world knowledge. For instance, Voyager [21] demonstrates continual learning in Minecraft through exploration, building a skill library, and iterative prompting that incorporates feedback, execution errors, and self-verification.

World model and policy: An RL agent simultaneously learns a world model and a behavioral policy. The world model defines possible environmental states, the agent's state, and transitions between states. The policy dictates which action an agent should take in a given state. Both the world model and the policy are learned from past experiences and received feedback.

Exploration and Exploitation: Unlike classical supervised or unsupervised learning where data is given, RL agents actively influence data collection through novel actions and observations—i.e., exploration. Agents must balance exploration (acquiring knowledge) with exploitation (applying existing knowledge to perform tasks). RL agents typically execute a sequence of actions to reach goals, each involving perception, reasoning, and action.

RL and Agentic AI: There is no one-to-one correspondence between RL agents and agentic AI systems. Agentic AI targets broader, more open-ended tasks and may define its own objectives. An RL agent aims to optimize a (more narrow) reward function by learning a policy for how to act in a specific environment. Nonetheless, Agentic AI incorporates many RL concepts, especially during training [113]. RL architectures like actor-critic models—which separate decision-making (actor) from evaluation (critic), often via self-assessment—are also commonly adopted in Agentic AI systems.

3.4.2 Interacting in (Virtual) Worlds

Often agents operate in restricted or digital environments though attempts have been made to consider general environments moving towards models that consider the world in its entirety or at least narrow tasks involving the physical world (or a model of it). For example, agents have also been employed in specialized simulation environments for robotics [157].

Multi-modal models that are trained on text, images and videos might constitute a potential world model [158]. That is, such models should, for example, given an image of the current state (such as a plane) and a textual description of an action (the plane will land soon) predict the next state (showing a plane that has landed). [159] deploys an agent in a model of a virtual home, which simulates the physical world. The agent performs random exploration and goal-oriented planning to gain experience, which are used to fine-tune an

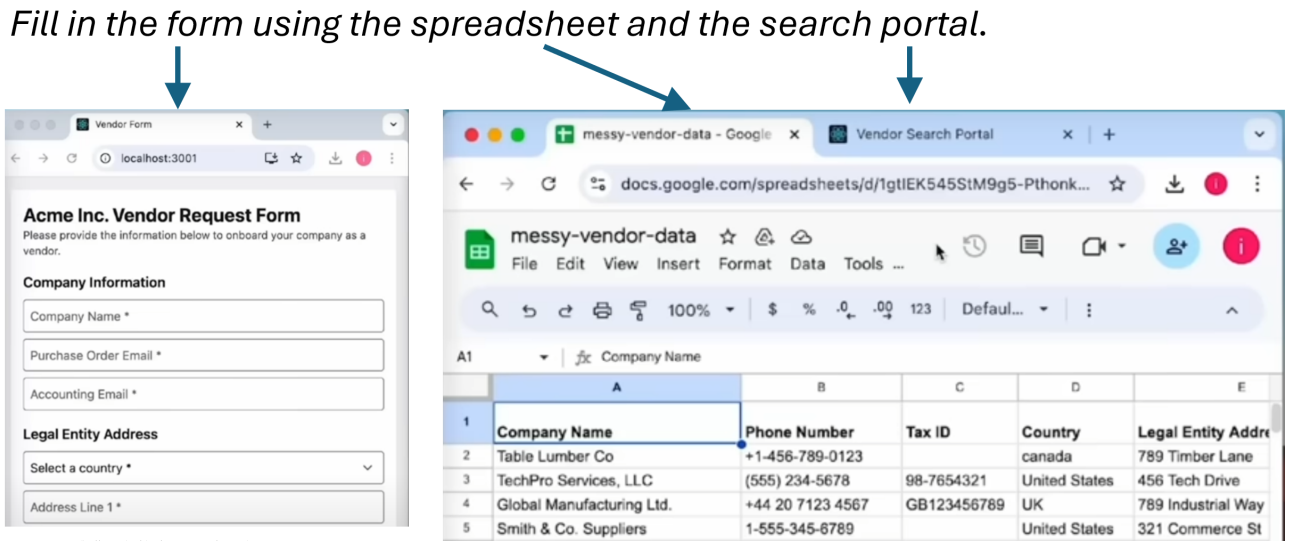

Fig. 12 Instruction to (Anthropic's) agent controlling a computer

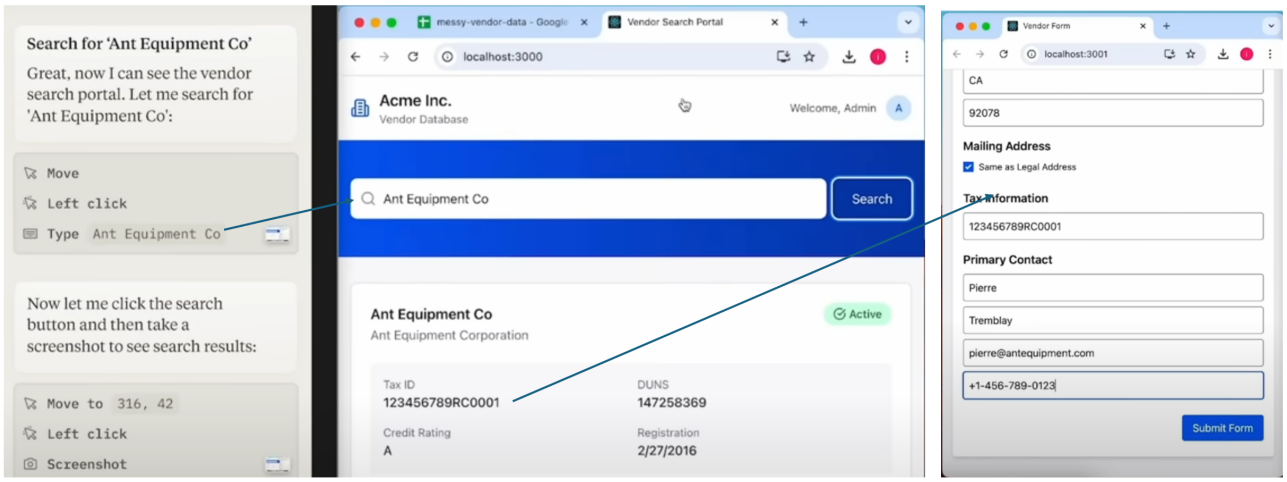

Fig. 13 (Anthropic's) agent controlling a computer through mouse, keyboard and sensing through screenshots as illustrated in the left panel

LLM. The fine-tuned LLM performs significantly better than much larger non-fine-tuned LLMs. Due to their extensive knowledge LLMs can be used as both a world model and a reasoning agent engaging in planning [100]. Here the reasoning agent builds a reasoning tree suggesting actions to solve a task under the guidance of the world model providing rewards.

While ordinary generative vision-language models allow to textually describe images and generate images from text. Agentic AI seeks to go beyond that by inferring actions given images and text. In particular, [160] developed a vision-language-action model by training a vision-language model on trajectories of robots containing both their observations and actions visual language tasks such as visual question answering.

Motion planing and 3D: While such approaches might improve significantly on prior work for the considered scenario, they still fall short in many aspects and tasks that are innate for humans. For example, they cannot perform 3D spatial reasoning such as estimating distances or size differences between objects without extensive training on this task [161].

The idea to leverage LLMs for motion planning, i.e., planning driving trajectories has also been explored [162]. They described a visual scene in text by specifying objects and their coordinates, which allowed a language model to process and reason upon the geometrical problem and output the motion trajectory.

Agents in computer games: Agents are commonly evaluated in computer games as they can be seen as open-worlds. [163] uses two phases to play the strategy game "MineCraft": i) an LLM to plan the behavior of an RL agent by defining subgoals, ii) an RL agent learning a policy for each subgoal and updating the LLM's world model [164] proposed an interactive planning approach for MineCraft based on describe, explain, plan, and select to yield feasible plans and reduce errors through self-explanation.

Interacting with browsers and computers: The idea to control a computer or at least the browser has gained increased attention. Commercial companies like OpenAI and Anthropic [165] have already released agents

performing simple tasks as illustrated in Figure 12, where the user prompt is addressed by controlling the mouse, keyboard and taking screenshots 13. Furthermore, a number of benchmarks where agents are supposed to solve tasks. To this end, multi-modal models can be used to extract information relevant to control the system from screenshots, e.g., as done with Omniparser V2 [166].

A web-browser can be seen as a tool that provides access to web-services but also as an environment to solve tasks. Systems with reasoning and planning capabilities can be enhanced with acting capabilities, e.g., [167], to make use of web-browsers [29, 30, 168]. However, as of now performing "relatively" simple tasks for humans, e.g., related to online shopping remains a challenge for AI agents [169]. More concretely, on the Workarena++ benchmark [168] humans score close to 100% while LLMs such GPT-4o and LLama3 score close to 0%. An example task is shown in Figure 14.

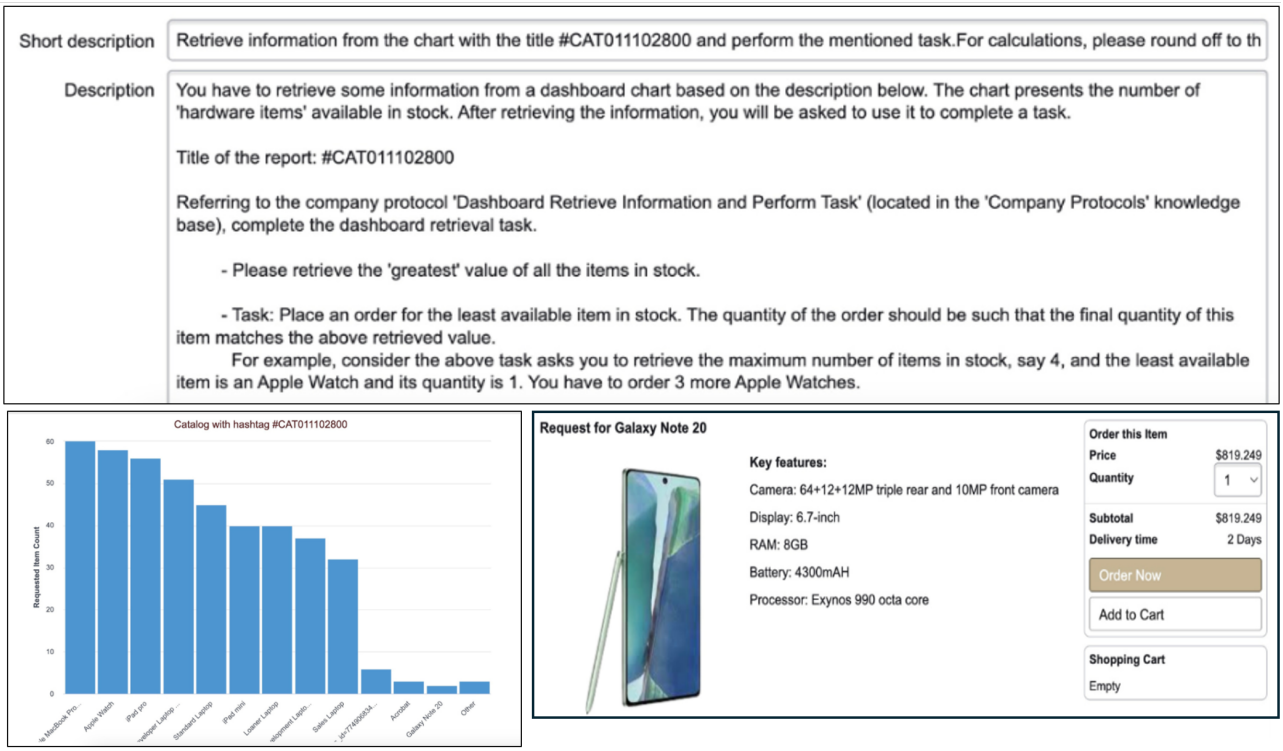

Fig. 14 Task from Workarena++ [168]. Given a ticket asking to refill inventory below four items (top panel), the system must understand a chart to identify items to order(left lower panel) and interact with an ordering page (right lower panel).

While the idea of agents controlling a computer with all its software and any further tuning towards agent usage is appealing, designing specialized agent friendly interfaces can improve an agent's success rate as demonstrated for software engineering [170].

Interacting with tools: Conceptually, tool usage (and access to external memory like databases) could also be viewed as environmental interaction. However, we treat both as integral components of the Agentic system. First, memory and tool usage often do not require a long sequence repeating perceiving, reasoning, acting, but rather tool usage is a single API call, e.g., to a database. Tool usage is often learnt in a supervised manner, e.g., by using samples of problem descriptions and the corresponding API calls. Second, we view an agent as being equipped with at least basic capabilities to use a tool. Reasoning might be needed to apply them successfully for a task if no supervised fine-tuning is performed. But if an agent lacks knowledge on tool usage, an agent can often rely on documentation, usage samples, and obtain immediate feedback, which tends to limit the needed interaction. However, especially for complex tools where no training data exists, a reinforcement learning setting might be the better conceptual match.

3.4.3 Interacting with Humans

Humans can assist, collaborate, or oversee the agent.

Scientific Processes: The AI scientist [171] and follow-up works [172] implement an end-to-end LLM driven scientific discovery process including idea generation, experiment iteration and paper write-up showing that it can generate 100s of medium-quality papers within a week. Deep Research by OpenAI and Google also support researchers by generating research reports, e.g., searching and summarizing works from a particular area or about a particular topic using multiple rounds of search and analysis. In particular, these commercial tools also often ask humans for clarifications as the research process takes a significant amount of time due to a

large computational demand. In the academic world, human-agent collaboration for research has been discussed in [173]. More specialized research agents have been proposed, e.g., for drug discovery [174]. Specialized agents can perform highly domain specific tasks, such as simulating molecules relevant to chemistry and modeling ecosystems as needed in biology [175].

Agents as Assistants: Agents can be used to support the execution of tasks but also for improving skills, e.g., practicing negotiations [176]. Despite an extensive number of already deployed applications, the optimal design of agents engaging with humans is still subject to research. For instance, recently back-channeling as an active listening strategy, where the LLM would utter phrases like "really?" or "Wow" at appropriate times led to higher conversational engagement [177].

Many areas aside from the aforementioned "scientific discovery" such as law, finance, psychology, education, medicine and military also benefit from Agentic AI with numerous applications [175]. For example, an Agentic AI workflow has been proposed to translate formal medical reports into patient-accessible reports reducing errors and hallucination through reflection [178]. Counseling agents for students being bullied have also been assessed [179] mainly by comparing existing commercial LLMs. For more medical examples consult [180]. Agentic AI has also been suggested as ethical counsel in the practice of law [181]. In finance, multi-agent systems have been proposed for decision making [182] as well as for trading [183].

3.4.4 Interacting with Other Agents

Agents can assume various roles: autonomously simulating aspects of society or systems, acting as integrated analytical components like critics or evaluators within reasoning processes, or collaborating independently to solve complex tasks through coordinated efforts.

Agents for Simulation: Agents have been used for societal and economic simulations, such as modeling macroeconomic dynamics with diverse interacting agents [184]. Agents have also been employed to simulate disease spread, such as during the COVID-19 pandemic [185]. In recommender systems, agents simulate both users and items to model interactions and improve recommendation quality [186]. Agents have also represented countries in simulations of historical wars [187].

Agents as Analytical Components: Having two LLM agents debate under the moderation of an LLM judge has fostered more divergent thinking [188] and improved summarization showin in benchmarks [189]. However, agents have also shown to reduce diversity by converging towards human-like polarization [190]. Additionally, incorrect or manipulated knowledge can spread rapidly within agent networks [191].

Agents as Independent Collaborators: To solve user-defined tasks, [192] proposed using role-playing agents, such as a Python programmer and a stock trader collaborating to develop a stock trading bot. Although these agents work together autonomously, the paper suggests introducing critics to provide feedback and improve outcomes. The paper also emphasizes the importance of carefully crafted initial prompts and precise task specifications, potentially generated by an LLM. [193] proposes using a hierarchical structure of agents led by a central leader. In each layer, messages from previous layers are aggregated, producing an averaging effect that stabilizes responses. The leader provides feedback that subordinate agents use for improvement.

3.5 Specifying and Evaluating Agentic AI

A critical aspect of any system is its specification and ensuring that the system conforms to the specification and broader goals such as legal compliance and efficiency.

3.5.1 Specifying an Agent

Defining an agent typically involves describing identity information (e.g., name, age, personality), motivational drivers, professional roles [23], tool permissions, delegation rights, workflows, and interaction behavior with other agents—see Figure 16. Although agent descriptions can be elaborate, simple descriptions often suffice—for instance, in a multi-agent programming setting, agents were defined using only a professional role, a goal, and constraints [24]. The definition also depends on the intended level of autonomy 2; agents with partial autonomy—whether due to technical limitations or legal reasons—may require consent checkpoints where humans review outcomes and planned future actions [194].

Agent Design: Manual and Automatic (Data-Driven or Model-Based): Agents can be designed either manually or automatically. Automatic design can involve fitting to data or through model optimization involving, e.g., an evaluation function. [195] evaluates the automated design of agentic systems. This approach requires specifying a search space of potential agents, a search algorithm to explore the space, and an evaluation function to assess the quality of agents. A meta-agent programs new agents by generating code, potentially based on archived prior agents, and employs a reflection process to ensure novelty. Evolutionary algorithms using crossover and mutation have also been used for automatic agent creation [196]. Data fitting can be achieved using few-shot prompting with examples or through sampling techniques. For example, to emulate human participants in a user study, agent profiles can be generated by sampling traits from a distribution [197]. [198] manually

defined a few agent profiles, which were subsequently expanded into a larger agent pool with the assistance of an LLM.

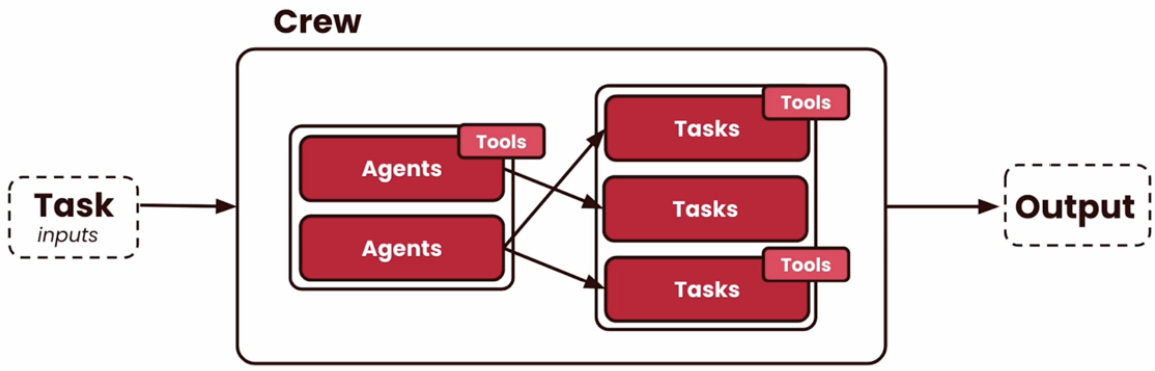

Fig. 15 Specifying a multi agents system using agents, tasks and their interconnection as abstractions specified by a user (Figure from [199])

```
planner = Agent(
    role="Content Planner",
    goal="Plan engaging and factually accurate content on {topic}",
    backstory="You're working on planning a blog article "
              "about the topic: {topic}."
              "You collect information that helps the "
              "audience learn something "
              "and make informed decisions. "
              "Your work is the basis for "
              "the Content Writer to write an
    allow_delegation=False,
    verbose=True
)
```

```
writer = Agent(
    role="Content Writer",
    goal="Write insightful and factually accurate "
         "opinion piece about the topic: {topic}",
    backstory="You're working on a writing "
              "a new opinion piece about the topic: {topic}. "
              "You base your writing on the work of "
              "the Content Planner, who provides an outline "
              "and relevant context about the topic. "
              "You follow the main objectives and "
              "direction of the outline, "
              "as provide by the Content Planner. "
              "You also provide objective and impartial insights "
              "and back them up with information "
              "provide by the Content Planner. "
              "You acknowledge in your opinion piece "
              "when your statements are opinions "
              "as opposed to objective statements.",
    allow_delegation=False,
    verbose=True
)
```

```
plan = Task(
    description=(
        "1. Prioritize the latest trends, key players
            "and noteworthy news on {topic}.\n"
        "2. Identify the target audience, considering
            "their interests and pain points.\n"
        "3. Develop a detailed content outline incluc
            "an introduction, key points, and a call
        "4. Include SEO keywords and relevant data or
    ),
    expected_output="A comprehensive content plan doc
        "with an outline, audience analysis, "
        "SEO keywords, and resources.",
    agent=planner,
)
```

Fig. 16 Example definition of a content planner agent, the content planning task and the writer agent providing procedural guidance including interaction with other agents (Figure from [199])

3.5.2 Specifying Multi-Agents

Agentic systems often comprise multiple agents. Using agents with different personas—potentially based on the same underlying LLM—can achieve better outcomes than chain-of-thought (CoT) prompting for tasks like creative writing and trivia [200], motivating the adoption of multiple agents. Multi-agent systems can be defined in various ways but generally follow similar structural patterns. Agents and tasks are defined independently, allowing flexible assignment of agents to tasks, as illustrated in Figure 15. Both agents and tasks can be flexibly specified through textual descriptions, similar to conventional prompting. An agent description mirrors that of a single-agent system, specifying goals, a role (or persona) characterizing attributes (e.g., empathetic behavior), allowed tool usage and delegation, constraints, and procedural task guidance. A task description may include expected outputs and procedural guidance, including instructions for agent interactions, as shown in Figure 15.

Multi-agent systems form distributed architectures where communication patterns and (self-)organization strategies, such as symmetry breaking [201], are crucial.

Key dimensions of Multi-Agents System: We provide essential dimensions for multi-agents systems synthesized from works that provide a more detailed discussion (e.g., [175, 202, 203])

- **Homogeneous vs. Heterogeneous:** Homogeneous agents can be exact replicas, meaning the underlying foundation model as well as the agent description are identical. Heterogeneous models can be based on different models, subject to different agent instructions.
- **Cooperative vs. Non-cooperative:** Cooperative agents pursue a shared goal and may use argumentative debate styles to enhance results [204].
- **Communication structure:** Communication can be classified as *push*—where messages are sent unsolicited—or *pull*—where agents request information explicitly. Communication can also be *centralized*—through a single agent or shared message pool [24]—or *decentralized*, where agents interact freely without a central node. Standardization of communication like the agent to agent protocol[205] facilitate interaction across diverse agents.
- **Organization (Flat vs. Hierarchical):** Agents can be organized in hierarchical settings, e.g., [193], or in a flat manner. For example, [206] employs multiple workers handling parts of a task (sequentially) and a manager aggregating results.
- **Learning:** Agents may learn independently in decentralized systems or share experiences centrally to accelerate collective improvement.
- **Communication content:** Communication is primarily text-based; cooperative systems often package instructions, goals, state descriptions, action histories, and dialogue histories into messages [207]. Agentic systems often mimic real-world development by featuring domain-expert agents that collaborate through discussions, potentially adopting varied communication styles [208].

3.5.3 Evaluating Agentic AI

Evaluating Agentic AI is challenging. Agents can behave non-deterministic with complex interactions and require a much longer time-span to complete tasks than typical for GenAI. Agents can exhibit a wide range of capabilities and desiderata that require evaluation. We outline key decision domains but refer readers to specialized surveys for further details [209]. Capabilities span general LLM skills like long-text understanding and instruction following, as well as agent-specific skills such as planning, learning from interactions, error handling, tool usage, and spatial reasoning [210]. Desiderata include task performance [210], failure awareness (recognizing impending failure), efficient tool usage, harmfulness [211], and computational efficiency—for instance, [212] enforced efficiency by setting computational limits for tasks. Security is also a key concern as agents come with their own threat model [213]. Beyond foundational capabilities, agents can also be evaluated on domain-specific competencies. For instance, evaluating agents simulating humans in user studies has involved manual comparisons with real users and sanity checks that vary names and gender to account for prompt sensitivity and model shortcomings [214].

Metrics can include generic measures such as completion rate (fraction of tasks completed) or task success rate [164], possibly supplemented by solution quality scores, self-aware failure rates (fraction of tasks where agents signal failure before failing), and tool call accuracy. In games, evaluation scores could reflect deviations from optimal actions (e.g., in rock-paper-scissors) or achievements awarded by the game engine [210].

Benchmarks designed for Generative AI, such as the well-known MMLU benchmark [13], can also be applied to Agentic AI; MMLU has been extended with more complex questions to better evaluate reasoning abilities [215]. Various datasets exist for evaluating (or training) Agentic AI, often focusing on specific areas such as tool usage (over 14,000 REST APIs [153]), robot learning [216], assistant-user task pairs [192], programming [217], social game playing [218], and simulating human behavior in user studies [214]. Frameworks are also available to facilitate task evaluation. [219] offers infrastructure enabling researchers to easily design custom evaluation tasks. Games are frequently used for evaluation, ranging from simple games like rock-paper-scissors to complex environments like Minecraft [210]. Several benchmarks target evaluating agents in web environments [29, 220], focusing on completing basic human tasks [29].

4 Towards AGI?

4.1 Challenges Moving Towards AGI

Many challenges found in GenAI also apply to Agentic AI, though some become either more pronounced or mitigated. They hinder the further evolution to Artificial General Intelligence (AGI). These challenges also posit research opportunities.

Errors: Although reasoning reduces errors compared to Generative AI, as shown in benchmarks (Figure 4), the complexity and length of tasks in Agentic AI increase the risk of cumulative errors across steps [17].

Robustness: Foundation models are known to be highly prompt sensitive [18]. In turn, agents relying on such models generating their own prompts as part of the reasoning process, are also highly sensitive to original prompts and potentially also to minor changes in the environment.

Interpretability: Agentic AI emphasizes step-by-step reasoning presented in readable text, enhancing interpretability. However, Chain-of-Thought (CoT) reasoning may not always be faithful to the underlying decision process [221]. Moreover, GenAI [39] and deep learning broadly [222] continue to struggle with explainability. Additionally, Agentic AI is more complex than GenAI because it integrates components like planning algorithms, external memory, and tool usage.

Dynamic and Complex Environments: Agents are designed to operate in non-deterministic, complex environments. They interact with changing environments, select tools, and dynamically collaborate with other agents, sometimes in parallel.

Observability: The environment might not be fully observable, e.g., we cannot gather all information about our world. This is a common assumption in RL [19]. Furthermore, agents' behaviors and tool operations may not be fully transparent. For example, agents might access tools via APIs whose internal mechanisms are opaque.

Safety and Security: Agents are vulnerable to harmful behaviors, either through deliberate attacks [211] or systemic shortcomings. This risk is higher for Agentic AI systems compared to GenAI, due to their increased interaction in less controlled environments. For instance, agent capabilities like tool invocation can be exploited. Furthermore, multi-modal agents also face attacks on each modality, e.g., foundation models can be jailbroken through adversarial images [223]. Privacy concerns escalate as agents share information with other agents and tools. Even seemingly minor information—such as an agent's role as a psychological counselor—can represent a privacy breach.

Unpredictable Emergent Behaviors: LLMs have shown remarkable emergent behavior due to scale, e.g., in-context learning[15] or chain-of-thought reasoning [31]. Such behaviors are difficult to foresee in many ways – in fact, many capabilities of models might even be dormant and waiting to be discovered. While for LLMs most can be deemed positive – or at least posing no immediate threat, this might change with "agent swarms" interacting with the real-world, e.g., such agent swarms could put democracies at risk at an unprecedented scale as elaborated in a paper by a large number of reputable researchers [224].

Evaluation: Evaluating Agentic AI—assessing reliability, task performance, and potential harms—remains challenging and incomplete [17].

Human Alignment: Agents may engage in unforeseen or unethical actions in pursuit of their goals, contrary to human intent.

Controlling and Monitoring Agents: Controlling agents is challenging. Even when human or independent approvals are required for agent actions, anticipating the consequences of approvals or disapprovals remains difficult [17]. Because agents are dynamic, they may adopt unpredictable behaviors to achieve their goals.

Resource Allocation and Management: Managing resources, such as computational access, is more demanding as agents dynamically consume varying and potentially arbitrary amounts of information [175].

4.2 Agentic AI to AGI?

AGI refers to AI that transcends narrow domains and can generalize to new, unfamiliar situations. Current AGI benchmarks, such as [212], emphasize novel tasks requiring learning from few examples and limited computation, rendering brute-force approaches impractical. Agentic AI enables novel applications beyond those possible with GenAI. Tasks requiring more complex reasoning and interactions become more and more feasible. Currently, agents are restricted to relatively simple tasks due to technological and regulatory constraints, but this is expected to evolve. Agentic AI may represent the next step toward Artificial General Intelligence (AGI), potentially equaling or surpassing human intelligence. Incremental technological advances and scaling of existing Agentic AI systems might be sufficient to achieve AGI. However, progress may stall due to barriers like limited availability of training data [225]. Some researchers argue that mere scaling is insufficient, and fundamentally new approaches are required [226]. Consequently, the timeline for achieving AGI remains uncertain, as evidenced by the wide range of predictions from respected researchers. For instance, in 2018, Ray Kurzweil predicted a 50% chance of human-level AI by 2029, while Stuart Russell estimated 50–70 years, Yann LeCun 50–100 years, and Rodney Brooks approximately 180 years [227].

What happens if AGI is reached? Even decades before the current AI boom, there were speculations about the consequences of achieving AGI [228, 229]. It is arguably difficult to predict the outcomes once AI surpasses human intelligence. Researchers like Stuart Russell suggest that once AI reaches a "Kindergarten level," its improvement could accelerate at more than an exponential rate [227]. In particular, AGI has been said to posit an existential risk to humanity [229] posing challenges with respect to control and value alignment. According to the instrumental convergence thesis, AI systems might independently pursue goals like self-preservation and resource acquisition, potentially clashing with human interests. These existential risks go far beyond more tangible economic risks such as unemployment and potential erosion of wages [229].

5 Methodology and Related Work

5.1 Methodology and Scope

Scope and Target Audience: Our focus is on contrasting Agentic AI and GenAI and highlighting key innovations in the transition, targeting a broad audience of academics and industry professionals. The basic characteristics distinguishing GenAI from Agentic AI are intended to be accessible to a wide audience, while recent innovations in Agentic AI are tailored for a more technically focused readership. To this end, we adopt a high-level capability perspective aimed at general interest. We assume readers have basic familiarity with deep learning and Generative AI concepts, including foundation models and prompt engineering, as covered in prior works (e.g., [18]). In-depth technical knowledge, such as of the transformer architecture, is not our focus, as it is covered extensively in other works (e.g., [45]). While we address all relevant areas of Agentic AI, we refer readers to other works for more extensive coverage of specific aspects.

Research Methodology: We primarily followed the literature review methodology outlined by [230], incorporating several innovations to address the rapid growth of research in this field. Specifically, we began by conducting a meta-survey, searching Google Scholar for surveys on "Agentic AI," "LLM agents," and "Generative AI." This approach served three goals: (i) identifying key papers for forward and backward search (i.e., snowballing [230]), (ii) building on existing works to ensure conceptual completeness, and (iii) improving prior works by synthesizing different viewpoints and identifying gaps. We assessed more than 30 surveys, but identified five surveys on Agentic AI [175, 231–234] and two on Generative AI [47, 235], which were chosen due to quality, recency, comprehensive and non-overlap with other chosen surveys. The Agentic AI surveys were significantly more influential in shaping our work. We conducted an initial read-through of these manuscripts. We then developed an outline for our paper, including chapter structures and basic content, unifying prior works while omitting less relevant aspects and introducing novel ones. In a second iteration, we revisited the surveys to identify gaps in our conceptualization, refining our framework primarily through backward search. In a third iteration, we conducted forward searches, focusing on citations and publications in top venues like NIPS, ICLR, and ICML, while also considering new arxiv.org papers. We restricted our search to works from 2024 onward (up to October 2026), as older works were assumed to be captured in the selected surveys. Additionally, we performed targeted searches on Google Scholar for each subchapter, emphasizing recent surveys on topics like evaluation, reasoning, planning, and RAG. These sources were used to further refine the structure and content of our work. To ensure the inclusion of the most recent results, we incorporated works from arxiv.org and selected blog posts from reputable sources such as OpenAI and Huggingface. However, these additional sources were included only after basic quality assessments. Also, note that the initial version from 2025 was updated in two subsequent iterations.

5.2 Related Surveys

As stated in the research methodology, we relate most strongly five surveys on "Agentic AI" and "LLM Agents". Key differentiators include: (i) None of the surveys focused on distinguishing GenAI from Agentic AI, which forms the first part of our survey. (ii) None offered a clear and multi-angled definition of Agentic AI. (iii) None discussed autonomy levels and the motivation for Agentic AI—including a contrasting perspective to AGI—in comparable detail. The second part of our survey overlaps more with existing works, but we expand on important practical aspects such as (iv) defining agents and memory characteristics, and (v) offer a different conceptual framework. We believe that offering multiple, diverse perspectives on Agentic AI is highly valuable. We now discuss these surveys in greater depth and highlight how they differ from our work:

[231], surveying "LLM-based autonomous agents," adopts an architecture-centric view describing key components but lacks a detailed discussion on defining agents. Our work is more capability-focused and differs significantly in its conceptualization. Additionally, we emphasize distinguishing GenAI from Agentic AI.

[232] adopts a conceptualization emphasizing brain, perception, and action, whereas we focus on reasoning and interaction. They explore the historical notion of agents and the motivation for using LLMs, while we emphasize the differentiation between GenAI and Agentic AI.

[233] surveys "Agentic AI" by comparing it to traditional AI, including early rule-based systems. In our view, the traditional AI era began in the 1960s and ended with the breakthroughs of Generative AI, notably GPT-2/3 around 2019. While their comparison is highly valuable, it is of less interest towards readers already familiar with GenAI such as ChatGPT. Our survey targets these readers by starting with Generative AI and not covering the entire history of AI. As a result, our conceptualization differs significantly—for example, Table 1 in [233] and our Table 1 share only one common aspect.

[234] surveys "Agentic LLMs." Our work differs by contrasting Agentic AI directly against GenAI. Our conceptualization also differs: at the highest level, we emphasize two rather than three core capabilities; additionally, we view retrieval augmentation not as reasoning (as in Figure 3 of [234]), but rather as interaction with a tool, aligning with earlier works like [37]. Basic reasoning forms like chain-of-thought [31] do not require retrieval, and even more advanced reasoning typically only necessitates large context windows. We offer a broader set of

motivations for Agentic AI (compare Table 3 in our work to Chapter 1.4 in [234]) and discuss agent definitions in greater depth.

[175] surveys "LLM-based agents" by comparing RL and LLM agents, whereas we contrast Agentic AI against GenAI. Our conceptualization differs at multiple levels: at a high level, we focus on novel AI agent capabilities like reasoning and interaction, while [175] adheres closely to traditional RL agent concepts.

[236] compares "AI agents and Agentic AI" also touching on differences to GenAI. It is more focused on describing applications (like [237]), while we provide a more fine granular resolution of conceptual differences between generative and agentic AI discuss more extensively reasoning, interaction, and almost entirely missing aspects such as self-improving agents [2]. We account more for the historic and gradual evolution from generative to agentic AI (see, e.g., Figure 7). We also have different conceptual views, e.g., in our view aligned with modern AI agents (e.g., [2]) an AI agent is not limited to a single task execution (see Table 8 [236]).

6 Conclusion

Agentic AI is a major paradigm shift beyond Generative AI by introducing reasoning, interaction, and autonomy at a new scale. Our survey and conceptualization systematically contrasts Agentic AI and GenAI from multiple perspectives. We cover its technical foundations, practical specification, and open challenges. As Agentic AI evolves, understanding its capabilities, risks, and the nuances of agent specification becomes essential for both advancing research and ensuring responsible deployment.

Declarations

Conflict of Interest

The authors declare that they have no conflict of interest.

Ethical Approval

Not applicable.

Informed Consent

Not applicable.

Funding

No funding was received for conducting this study.

Data Availability Statement

Data sharing is not applicable to this article as no datasets were generated or analyzed during the current study. All references are included in the paper.

Author Contributions

Not applicable. There is just one, which did everything.

Acknowledgments

Not applicable.

References

[1] Liu, J., Zhao, X., Shang, X., Shen, Z.: Dive into claude code: The design space of today's and future ai agent systems. arXiv preprint arXiv:2604.14228 (2026)

[2] OpenClaw: Self-learning. https://docs.openclaw.ai/tools/self-learning. Accessed: 2026-10-17

[3] Chollet, F.: On the Measure of Intelligence (2019). https://doi.org/10.48550/arXiv.1911.01547 . https://arxiv.org/abs/1911.01547

[4] OpenAI: OpenAI and Hugging Face Partner to Address Security Incident During Model Evaluation. Accessed: 2026-10-16 (2026). https://openai.com/index/hugging-face-model-evaluation-security-incident/

[5] Vincent, J.: Top AI researchers and CEOs warn against risk of extinction in 22-word statement. The Verge. Accessed: 2026-10-16 (2023)

[6] Nolan, B.: Thousands of AI researchers were asked if tech could make humans extinct. A surprising number said yes. Business Insider. Accessed: 2025-04-16 (2024)

[7] Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., Tafjord, O.: Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457 (2018)

[8] El-Kishky, A., Wei, A., Saraiva, A., Minaiev, B., Selsam, D., Dohan, D., Song, F., Lightman, H., Clavera, I., Pachocki, J., Tworek, J., Kuhn, L., Kaiser, L., Chen, M., Schwarzer, M., Rohaninejad, M., McAleese, N., Mürk, O., Garg, R., Shu, R., Sidor, S., Kosaraju, V., Zhou, W., contributors: Competitive programming with large reasoning models. arXiv preprint arXiv:2502.06807 (2025)

[9] Zhong, T., Liu, Z., Pan, Y., Zhang, Y., Zhou, Y., Liang, S., Wu, Z., Lyu, Y., Shu, P., Yu, X., et al.: Evaluation of openai o1: Opportunities and challenges of agi. arXiv preprint arXiv:2409.18486 (2024)

[10] Marr, B.: Generative AI vs. Agentic AI: The Key Differences Everyone Needs to Know. https://www.forbes.com/sites/bernardmarr/2025/02/03/generative-ai-vs-agentic-ai-the-key-differences-everyone-needs-to-know/. Accessed: 2025-04-23 (2025)

[11] Hou, G., Zhang, W., Shen, Y., Tan, Z., Shen, S., Lu, W.: Entering real social world! benchmarking the theory of mind and socialization capabilities of llms from a first-person perspective. arXiv preprint arXiv:2410.06195 (2024)

[12] Chen, A., Song, Y., Zhu, W., Chen, K., Yang, M., Zhao, T., et al.: Evaluating o1-like llms: Unlocking reasoning for translation through comprehensive analysis. arXiv preprint arXiv:2502.11544 (2025)

[13] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., Steinhardt, J.: Measuring massive multitask language understanding. In: International Conference on Learning Representations (ICLR) (2021). https://arxiv.org/abs/2009.03300

[14] Paperswithcode: SOTA: Multi-task Language Understanding (MMLU). https://paperswithcode.com/sota/multi-task-language-understanding-on-mmlu. Accessed: 2025-04-22 (2025)

[15] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners. arXiv preprint arXiv:2005.14165 (2020)

[16] Pounds, E.: What Is Agentic AI? https://blogs.nvidia.com/blog/what-is-agentic-ai/. Accessed: 2025-04-20 (2024)

[17] Shavit, Y., Agarwal, S., Brundage, M., Adler, S., O'Keefe, C., Campbell, R., Lee, T., Mishkin, P., Eloundou, T., Hickey, A., Slama, K., Ahmad, L., McMillan, P., Vallone, A., Passos, A., Robinson, D.G.: Practices for Governing Agentic AI Systems. https://openai.com/index/practices-for-governing-agentic-ai-systems/. Accessed: 2025-04-23 (2023)

[18] Schneider, J., Meske, C., Kuss, P.: Foundation models: A new paradigm for artificial intelligence. Business & Information Systems Engineering **66**, 221–231 (2024)

[19] Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach, 4th edn. Pearson, Hoboken, NJ (2021)

[20] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems **36**, 8634–8652 (2023)

[21] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291 (2023)

[22] Agent.ai: AI Agents Explained. https://docs.agent.ai/ai-agents-explained. Accessed: 2025-04-24 (2025)

[23] Park, J.S., O'Brien, J., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative agents: Interactive simulacra of human behavior. In: Proceedings of the 36th Annual Acm Symposium on User Interface Software and Technology, pp. 1–22 (2023)

[24] Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., et al.: Metagpt: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 (2023)

[25] European Union: EU AI Act. https://artificialintelligenceact.eu/. Accessed: 2025-15-02 (2023)

[26] BBC News: Microsoft chatbot is taken offline after it learns to be racist. BBC News. Accessed: 2025-04-18 (2016)

[27] CDOTrends: Meta AI's New Chatbot Goes 'Bad' in Days (2022). https://www.cdotrends.com/story/16654/meta-ai%E2%80%99s-new-chatbot-goes-%E2%80%98bad%E2%80%99-days

[28] Reed, S., Zolna, K., Parisotto, E., Colmenarejo, S.G., Novikov, A., Barth-Maron, G., Gimenez, M., Sulsky, Y., Kay, J., Springenberg, J.T., et al.: A generalist agent. arXiv preprint arXiv:2205.06175 (2022)

[29] Drouin, A., Gasse, M., Caccia, M., Laradji, I.H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., et al.: Workarena: How capable are web agents at solving common knowledge work tasks? arXiv preprint arXiv:2403.07718 (2024)

[30] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., et al.: Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854 (2023)

[31] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., *et al.*: Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems **35**, 24824–24837 (2022)

[32] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. arXiv preprint arXiv:2001.08361 (2020)

[33] Chung, H.W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., *et al.*: Scaling instruction-finetuned language models. Journal of Machine Learning Research **25**(70), 1–53 (2024)

[34] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. Advances in neural information processing systems **36**, 11809–11822 (2023)

[35] Wang, Z., Tu, H., Zhang, L., Chen, H., Wu, J., et al.: Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw (2026). https://doi.org/10.48550/arXiv.2604.04759 . https://arxiv.org/abs/2604.04759

[36] Maynez, J., Narayan, S., Bohnet, B., McDonald, R.: On faithfulness and factuality in abstractive summarization. arXiv preprint arXiv:2005.00661 (2020)

[37] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., *et al.*: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)

[38] Dziri, N., Madotto, A., Zaïane, O., Bose, A.J.: Neural path hunter: Reducing hallucination in dialogue systems via path grounding. arXiv preprint arXiv:2104.08455 (2021)

[39] Schneider, J.: Explainable generative ai (genxai): A survey, conceptualization, and research agenda. Artificial Intelligence Review **57**(11), 289 (2024)

[40] Zoe Kleinmann: Why Google's 'woke' AI problem won't be an easy fix. https://www.bbc.com/news/technology-68412620. Accessed: 2025-04-23 (2024)

[41] Beeching, E., Tunstall, L., Rush, S.: Scaling test-time compute with open models. https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute. Accessed: 2025-06-30 (2024)

[42] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. OpenAI Technical Report (2019)

[43] OpenAI: ChatGPT: Optimizing Language Models for Dialogue. https://openai.com/blog/chatgpt/.

Accessed: 2025-04-15 (2022)

[44] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., *et al.*: Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems **35**, 27730–27744 (2022)

[45] Schneider, J.: What comes after transformers?–a selective survey connecting ideas in deep learning. arXiv preprint arXiv:2408.00386 (2024)

[46] Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. Advances in neural information processing systems **27** (2014)

[47] Zhang, C., Zhang, C., Zheng, S., Qiao, Y., Li, C., Zhang, M., Dam, S.K., Thwal, C.M., Tun, Y.L., Huy, L.L., et al.: A complete survey on generative ai (aigc): Is chatgpt from gpt-4 to gpt-5 all you need? arXiv preprint arXiv:2303.11717 (2023)

[48] Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. Advances in neural information processing systems **35**, 22199–22213 (2022)

[49] OpenAI: Introducing GPT-4.5. https://openai.com/index/introducing-gpt-4-5. Accessed: 2025-04-24 (2025)

[50] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021)

[51] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint arXiv:2302.11382 (2023)

[52] Schneider, J.: Agentic ai prompt engineering: Advancing generative ai patterns conceptually. Available at SSRN 5402177 (2025)

[53] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems **36**, 68539–68551 (2023)

[54] Anthropic: Introducing the Model Context Protocol. Accessed: 2026-10-17 (2024). https://www.anthropic.com/news/model-context-protocol

[55] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al.: Webgpt: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332 (2021)

[56] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR) (2023)

[57] Richards, T.B.: AutoGPT: An autonomous GPT-4 experiment. https://github.com/Torantulino/Auto-GPT. GitHub repository (2023)

[58] Wang, Z.M., Peng, Z., Que, H., Liu, J., Zhou, W., Wu, Y., Guo, H., Gan, R., Ni, Z., Yang, J., et al.: Rolellm: Benchmarking, eliciting, and enhancing role-playing abilities of large language models. arXiv preprint arXiv:2310.00746 (2023)

[59] Lee, S., Park, S.H., Kim, S., Seo, M.: Aligning to thousands of preferences via system message generalization. Advances in Neural Information Processing Systems **37**, 73783–73829 (2024)

[60] OpenAI: Introducing the GPT Store. https://openai.com/blog/introducing-the-gpt-store. Accessed: 2026-10-16 (2024)

[61] Oxford Languages: Definition of reasoning. https://languages.oup.com/. Accessed: 2025-04-15 (2025)

[62] Yasunaga, M., Chen, X., Li, Y., Pasupat, P., Leskovec, J., Liang, P., Chi, E.H., Zhou, D.: Large language models as analogical reasoners. arXiv preprint arXiv:2310.01714 (2023)

[63] Lin, B.Y., Fu, Y., Yang, K., Brahman, F., Huang, S., Bhagavatula, C., Ammanabrolu, P., Choi, Y., Ren, X.: Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. Advances in Neural Information Processing Systems **36**, 23813–23825 (2023)

[64] Mondorf, P., Plank, B.: Beyond accuracy: Evaluating the reasoning behavior of large language models – a survey. arXiv preprint arXiv:2404.01869 (2024)

[65] Sprague, Z., Yin, F., Rodriguez, J.D., Jiang, D., Wadhwa, M., Singhal, P., Zhao, X., Ye, X., Mahowald, K., Durrett, G.: To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning. arXiv preprint arXiv:2409.12183 (2024)

[66] Menick, J., Trebacz, M., Mikulik, V., Aslanides, J., Song, F., Chadwick, M., Glaese, M., Young, S., Campbell-Gillingham, L., Irving, G., et al.: Teaching language models to support answers with verified quotes. arXiv preprint arXiv:2203.11147 (2022)

[67] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., Neubig, G.: Pal: Program-aided language models. In: International Conference on Machine Learning, pp. 10764–10799 (2023). PMLR

[68] Chen, W., Ma, X., Wang, X., Cohen, W.W.: Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. arXiv preprint arXiv:2211.12588 (2022)

[69] Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q., et al.: Least-to-most prompting enables complex reasoning in large language models. arXiv preprint arXiv:2205.10625 (2022)

[70] Wang, X., Zhou, D.: Chain-of-thought reasoning without prompting. arXiv preprint arXiv:2402.10200 (2024)

[71] Li, D., Cao, S., Griggs, T., Liu, S., Mo, X., Tang, E., Hegde, S., Hakhamaneshi, K., Patil, S.G., Zaharia, M., et al.: Llms can easily learn to reason from demonstrations structure, not content, is what matters! arXiv preprint arXiv:2502.07374 (2025)

[72] Shen, S., Huang, F., Zhao, Z., Liu, C., Zheng, T., Zhu, D.: Long is more important than difficult for training reasoning models. arXiv preprint arXiv:2503.18069 (2025)

[73] Xu, S., Xie, W., Zhao, L., He, P.: Chain of draft: Thinking faster by writing less. arXiv preprint arXiv:2502.18600 (2025) arXiv:2502.18600 [cs.CL]

[74] Ajay, A., Han, S., Du, Y., Li, S., Gupta, A., Jaakkola, T., Tenenbaum, J., Kaelbling, L., Srivastava, A., Agrawal, P.: Compositional foundation models for hierarchical planning. Advances in Neural Information Processing Systems **36**, 22304–22325 (2023)

[75] Yang, L., Yu, Z., Zhang, T., Cao, S., Xu, M., Zhang, W., Gonzalez, J.E., Cui, B.: Buffer of thoughts: Thought-augmented reasoning with large language models. Advances in Neural Information Processing Systems **37**, 113519–113544 (2024)

[76] Wu, J., Feng, M., Zhang, S., Che, F., Wen, Z., Tao, J.: Beyond examples: High-level automated reasoning paradigm in in-context learning via mcts. arXiv preprint arXiv:2411.18478 (2024)

[77] Yang, L., Yu, Z., Cui, B., Wang, M.: Reasonflux: Hierarchical llm reasoning via scaling thought templates. arXiv preprint arXiv:2502.06772 (2025)

[78] Wang, Y., Zhao, S., Wang, Z., Huang, H., Fan, M., Zhang, Y., Wang, Z., Wang, H., Liu, T.: Strategic chain-of-thought: Guiding accurate reasoning in llms through strategy elicitation. arXiv preprint arXiv:2409.03271 (2024)

[79] Zhou, P., Pujara, J., Ren, X., Chen, X., Cheng, H.-T., Le, Q.V., Chi, E., Zhou, D., Mishra, S., Zheng, H.S.: Self-discover: Large language models self-compose reasoning structures. Advances in Neural Information Processing Systems **37**, 126032–126058 (2024)

[80] Gao, P., Xie, A., Mao, S., Wu, W., Xia, Y., Mi, H., Wei, F.: Meta reasoning for large language models. arXiv preprint arXiv:2406.11698 (2024)

[81] Boud, D., Keogh, R., Walker, D.: Reflection: Turning Experience Into Learning. Routledge, London (1985)

[82] Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D.: Large language models cannot self-correct reasoning yet. In: The Twelfth International Conference on Learning Representations (2024)

[83] Zhang, W., Shen, Y., Wu, L., Peng, Q., Wang, J., Zhuang, Y., Lu, W.: Self-contrast: Better reflection through inconsistent solving perspectives. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 3602–3622 (2024)

[84] Chen, X., Lin, M., Schärli, N., Zhou, D.: Teaching large language models to self-debug. arXiv preprint arXiv:2304.05128 (2023)

[85] Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., Tian, Y.: Training large language models to reason in a continuous latent space. arXiv preprint arXiv:2412.06769 (2024)

[86] Chen, H., Feng, Y., Liu, Z., Yao, W., Prabhakar, A., Heinecke, S., Ho, R., Mui, P., Savarese, S., Xiong, C., et al.: Language models are hidden reasoners: Unlocking latent reasoning capabilities via self-rewarding. arXiv preprint arXiv:2411.04282 (2024)

[87] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2022)

[88] Imani, S., Du, L., Shrivastava, H.: Mathprompter: Mathematical reasoning using large language models. arXiv preprint arXiv:2303.05398 (2023)

[89] Razghandi, A., Hosseini, S.M.H., Baghshah, M.S.: Cer: Confidence enhanced reasoning in llms. arXiv preprint arXiv:2502.14634 (2025)

[90] Hosseini, A., Yuan, X., Malkin, N., Courville, A., Sordoni, A., Agarwal, R.: V-star: Training verifiers for self-taught reasoners. arXiv preprint arXiv:2402.06457 (2024)

[91] Zheng, C., Liu, Z., Xie, E., Li, Z., Li, Y.: Progressive-hint prompting improves reasoning in large language models. arXiv preprint arXiv:2304.09797 (2023)

[92] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., *et al.*: Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems **36**, 46534–46594 (2023)

[93] Liu, L., Zhang, C., Wu, L., Zhao, C., Hu, Z., He, M., Fan, J.: Instruct-of-reflection: Enhancing large language models iterative reflection capabilities via dynamic-meta instruction. arXiv preprint arXiv:2503.00902 (2025)

[94] Bi, Z., Han, K., Liu, C., Tang, Y., Wang, Y.: Forest-of-thought: Scaling test-time compute for enhancing llm reasoning. arXiv preprint arXiv:2412.09078 (2024)

[95] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., *et al.*: Graph of thoughts: Solving elaborate problems with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 17682–17690 (2024)

[96] Gandhi, K., Lee, D., Grand, G., Liu, M., Cheng, W., Sharma, A., Goodman, N.D.: Stream of search (sos): Learning to search in language. arXiv preprint arXiv:2404.03683 (2024)

[97] Schultz, J., Adamek, J., Jusup, M., Lanctot, M., Kaisers, M., Perrin, S., Hennes, D., Shar, J., Lewis, C., Ruoss, A., et al.: Mastering board games by external and internal planning with language models. arXiv preprint arXiv:2412.12119 (2024)

[98] Lingam, V., Tehrani, B.O., Sanghavi, S., Gupta, G., Ghosh, S., Liu, L., Huan, J., Deoras, A.: Enhancing language model agents using diversity of thoughts. In: The Thirteenth International Conference on Learning Representations (2025)

[99] Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., Stone, P.: Llm+ p: Empowering large language models with optimal planning proficiency. arXiv preprint arXiv:2304.11477 (2023)

[100] Hao, S., Gu, Y., Ma, H., Hong, J.J., Wang, Z., Wang, D.Z., Hu, Z.: Reasoning with language model is planning with world model. arXiv preprint arXiv:2305.14992 (2023)

[101] Hazra, R., Dos Martires, P.Z., De Raedt, L.: Saycanpay: Heuristic planning with large language models using learnable domain knowledge. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 20123–20133 (2024)

[102] Pallagani, V., Muppasani, B.C., Roy, K., Fabiano, F., Loreggia, A., Murugesan, K., Srivastava, B., Rossi, F., Horesh, L., Sheth, A.: On the prospects of incorporating large language models (llms) in automated planning and scheduling (aps). In: Proceedings of the International Conference on Automated Planning and Scheduling, vol. 34, pp. 432–444 (2024)

[103] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)

[104] AI, M.: Llama 4: Multimodal Intelligence. Accessed: 2025-04-23 (2025). https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[105] Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., *et al.*: The Llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024) https://doi.org/10.48550/arXiv.2407.21783

[106] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (long and Short Papers), pp. 4171–4186 (2019)

[107] Schneider, J.: Improving next tokens via second-last predictions with generate and refine. In: Intelligent Data Analysis (IDA) (2025)

[108] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)

[109] Zelikman, E., Wu, Y., Mu, J., Goodman, N.: Star: Bootstrapping reasoning with reasoning. Advances in Neural Information Processing Systems **35**, 15476–15488 (2022)

[110] Zhang, H., Chen, J., Jiang, F., Yu, F., Chen, Z., Li, J., Chen, G., Wu, X., Zhang, Z., Xiao, Q., et al.: Huatuogpt, towards taming language model to be a doctor. arXiv preprint arXiv:2305.15075 (2023)

[111] Chen, X., Xu, J., Liang, T., He, Z., Pang, J., Yu, D., Song, L., Liu, Q., Zhou, M., Zhang, Z., et al.: Do not think that much for 2+ 3=? on the overthinking of o1-like llms. arXiv preprint arXiv:2412.21187 (2024)

[112] Sui, Y., Chuang, Y.-N., Wang, G., Zhang, J., Zhang, T., Yuan, J., Liu, H., Wen, A., Zhong, S., Zou, N., et al.: Stop overthinking: A survey on efficient reasoning for large language models. arXiv preprint arXiv:2503.16419 (2025)

[113] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)

[114] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 (2024)

[115] Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. Advances in Neural Information Processing Systems **36**, 53728–53741 (2023)

[116] Ziegler, D.M., Stiennon, N., Wu, J., Brown, T.B., Radford, A., Amodei, D., Christiano, P., Irving, G.: Fine-tuning language models from human preferences. arXiv preprint arXiv:1909.08593 (2019)

[117] Liu, H., Sferrazza, C., Abbeel, P.: Chain of hindsight aligns language models with feedback. arXiv preprint arXiv:2302.02676 (2023)

[118] Dubois, Y., Li, C.X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P.S., Hashimoto, T.B.: Alpacafarm: A simulation framework for methods that learn from human feedback. Advances in Neural Information Processing Systems **36**, 30039–30069 (2023)

[119] Ivison, H., Wang, Y., Liu, J., Wu, Z., Pyatkin, V., Lambert, N., Smith, N.A., Choi, Y., Hajishirzi, H.: Unpacking dpo and ppo: Disentangling best practices for learning from preference feedback. Advances in neural information processing systems **37**, 36602–36633 (2024)

[120] Zhang, D., Zhoubian, S., Hu, Z., Yue, Y., Dong, Y., Tang, J.: Rest-mcts*: Llm self-training via process reward guided tree search. Advances in Neural Information Processing Systems **37**, 64735–64772 (2024)

[121] Schneider, J., Prabhushankar, M.: Understanding and leveraging the learning phases of neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 14886–14893 (2024)

[122] French, R.M.: Catastrophic forgetting in connectionist networks. Trends in cognitive sciences **3**(4), 128–135 (1999)

[123] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., *et al.*: Overcoming catastrophic forgetting in neural networks. Proceedings of the national academy of sciences **114**(13), 3521–3526 (2017)

[124] Miller, G.A.: The magical number seven, plus or minus two: Some limits on our capacity for processing information. Psychological Review **63**(2), 81–97 (1956) https://doi.org/10.1037/h0043158

[125] Team, G., Georgiev, P., Lei, V.I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al.: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530 (2024)

[126] Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E.H., Schärli, N., Zhou, D.: Large language models can be easily distracted by irrelevant context. In: International Conference on Machine Learning, pp. 31210–31227 (2023). PMLR

[127] Packer, C., Fang, V., Patil, S., Lin, K., Wooders, S., Gonzalez, J.: Memgpt: Towards llms as operating systems. arXiv preprint arXiv:2310.08560 (2023)

[128] Wang, Z.Z., Mao, J., Fried, D., Neubig, G.: Agent workflow memory. arXiv preprint arXiv:2409.07429 (2024)

[129] Zhang, Q., Hu, C., Upasani, S., Ma, B., Hong, F., Kamanuru, V., Rainton, J., Wu, C., Ji, M., Li, H., Thakker, U., Zou, J., Olukotun, K.: Agentic context engineering: Evolving contexts for self-improving language models. In: International Conference on Learning Representations (2026). https://arxiv.org/abs/2510.04618

[130] Kang, M., Chen, W.-N., Han, D., Inan, H.A., Wutschitz, L., Chen, Y., Sim, R., Rajmohan, S.: ACON: Optimizing context compression for long-horizon LLM agents. In: Proceedings of the 43rd International Conference on Machine Learning (2026)

[131] Yan, S., Yang, X., Huang, Z., Nie, E., Ding, Z., Li, Z., Ma, X., Bi, J., Kersting, K., Pan, J.Z., Schuetze, H., Tresp, V., Ma, Y.: Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (2026)

[132] Zhang, J., Hu, S., Lu, C., Lange, R., Clune, J.: Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. Version 3, revised 12 March 2026 (2025). https://arxiv.org/abs/2505.22954

[133] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-rag: Learning to retrieve, generate, and critique through self-reflection. In: The International Conference on Learning Representations (2023)

[134] Zhang, T., Patil, S.G., Jain, N., Shen, S., Zaharia, M., Stoica, I., Gonzalez, J.E.: Raft: Adapting language model to domain specific rag. In: First Conference on Language Modeling (2024)

[135] Yan, S.-Q., Gu, J.-C., Zhu, Y., Ling, Z.-H.: Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884 (2024)

[136] Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., Huang, G.: Expel: Llm agents are experiential learners. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 19632–19642 (2024)

[137] Zheng, L., Wang, R., Wang, X., An, B.: Synapse: Trajectory-as-exemplar prompting with memory for computer control. arXiv preprint arXiv:2306.07863 (2023)

[138] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (2023)

[139] Chen, M., Li, T., Sun, H., Zhou, Y., Zhu, C., Wang, H., Pan, J.Z., Zhang, W., Chen, H., Yang, F., Zhou, Z., Chen, W.: Research: Learning to reason with search for llms via reinforcement learning. arXiv preprint arXiv:2503.19470 (2025) arXiv:2503.19470 [cs.AI]

[140] Li, Z., Li, C., Zhang, M., Mei, Q., Bendersky, M.: Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 881–893 (2024)

[141] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., *et al.*: Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. Advances in Neural Information Processing Systems **36**, 42330–42357 (2023)

[142] Karpukhin, V., Oguz, B., Min, S., Lewis, P.S., Wu, L., Edunov, S., Chen, D., Yih, W.-t.: Dense passage retrieval for open-domain question answering. In: EMNLP (1), pp. 6769–6781 (2020)

[143] Zhang, Q., Chen, S., Bei, Y., Yuan, Z., Zhou, H., Hong, Z., Dong, J., Chen, H., Chang, Y., Huang, X.: A survey of graph retrieval-augmented generation for customized large language models. arXiv preprint arXiv:2501.13958 (2025)

[144] Singh, A., Ehtesham, A., Kumar, S., Khoei, T.T.: Agentic retrieval-augmented generation: A survey on agentic rag. arXiv preprint arXiv:2501.09136 (2025)

[145] Wölflein, G., Ferber, D., Truhn, D., Arandjelović, O., Kather, J.N.: Llm agents making agent tools. arXiv preprint arXiv:2502.11705 (2025)

[146] Yuan, L., Chen, Y., Wang, X., Fung, Y.R., Peng, H., Ji, H.: Craft: Customizing llms by creating and retrieving from specialized toolsets. arXiv preprint arXiv:2309.17428 (2023)

[147] Cai, T., Wang, X., Ma, T., Chen, X., Zhou, D.: Large language models as tool makers. arXiv preprint arXiv:2305.17126 (2023)

[148] Ding, H., Tao, S., Pang, L., Wei, Z., Gao, J., Ding, B., Shen, H., Cheng, X.: Toolcoder: A systematic code-empowered tool learning framework for large language models. arXiv preprint arXiv:2502.11404 (2025)

[149] Ruan, J., Chen, Y., Zhang, B., Xu, Z., Bao, T., Du, G., Shi, S., Mao, H., Li, Z., Zeng, X., Zhao, R.: TPTU: Task planning and tool usage of large language model-based AI agents. In: NeurIPS Workshop on Foundation Models for Decision Making (2023). https://openreview.net/forum?id=GrkgKtOjaH

[150] Chen, Z., Zhou, K., Zhang, B., Gong, Z., Zhao, W.X., Wen, J.-R.: Chatcot: Tool-augmented chain-of-thought reasoning on chat-based large language models. arXiv preprint arXiv:2305.14323 (2023)

[151] Parisi, A., Zhao, Y., Fiedel, N.: Talm: Tool augmented language models. arXiv preprint arXiv:2205.12255 (2022)

[152] Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. Advances in Neural Information Processing Systems **37**, 126544–126565 (2024)

[153] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al.: Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789 (2023)

[154] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable code actions elicit better llm agents. In: Forty-first International Conference on Machine Learning (2024)

[155] Qiao, B., Li, L., Zhang, X., He, S., Kang, Y., Zhang, C., Yang, F., Dong, H., Zhang, J., Wang, L., Ma, M., Zhao, P., Qin, S., Qin, X., Du, C., Xu, Y., Lin, Q., Rajmohan, S., Zhang, D.: Taskweaver: A code-first agent framework. arXiv preprint arXiv:2311.17541 (2023)

[156] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. Advances in Neural Information Processing Systems **36**, 38154–38180 (2023)

[157] Wang, J., Shi, E., Hu, H., Ma, C., Liu, Y., Wang, X., Yao, Y., Liu, X., Ge, B., Zhang, S.: Large language models for robotics: Opportunities, challenges, and perspectives. Journal of Automation and Intelligence (2024)

[158] Ge, Z., Huang, H., Zhou, M., Li, J., Wang, G., Tang, S., Zhuang, Y.: Worldgpt: Empowering llm as multimodal world model. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 7346–7355 (2024)

[159] Xiang, J., Tao, T., Gu, Y., Shu, T., Wang, Z., Yang, Z., Hu, Z.: Language models meet world models: Embodied experiences enhance language models. Advances in neural information processing systems **36**, 75392–75412 (2023)

[160] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al.: Rt-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818 (2023)

[161] Chen, B., Xu, Z., Kirmani, S., Ichter, B., Sadigh, D., Guibas, L., Xia, F.: Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 14455–14465 (2024)

[162] Mao, J., Ye, J., Qian, Y., Pavone, M., Wang, Y.: A language agent for autonomous driving. arXiv preprint arXiv:2311.10813 (2023)

[163] Nottingham, K., Ammanabrolu, P., Suhr, A., Choi, Y., Hajishirzi, H., Singh, S., Fox, R.: Do embodied agents dream of pixelated sheep: Embodied decision making using language guided world modelling. In: International Conference on Machine Learning, pp. 26311–26325 (2023). PMLR

[164] Wang, Z., Cai, S., Chen, G., Liu, A., Ma, X., Liang, Y.: Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. arXiv preprint arXiv:2302.01560 (2023)

[165] Anthropic: Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku. Anthropic News (2024). Accessed: 2025-04-09

[166] Lu, Y., Dhome-Casanova, T., Yang, J., Awadallah, A.: OmniParser V2: Turning Any LLM into a Computer Use Agent. Microsoft Research (2025). https://www.microsoft.com/en-us/research/articles/omniparser-v2-turning-any-llm-into-a-computer-use-agent/

[167] Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.-X.: Language agent tree search unifies reasoning acting and planning in language models. arXiv preprint arXiv:2310.04406 (2023)

[168] Boisvert, L., Thakkar, M., Gasse, M., Caccia, M., Chezelles, T., Cappart, Q., Chapados, N., Lacoste, A., Drouin, A.: Workarena++: Towards compositional planning and reasoning-based common knowledge work tasks. Advances in Neural Information Processing Systems **37**, 5996–6051 (2024)

[169] Jin, Y., Li, Z., Zhang, C., Cao, T., Gao, Y., Jayarao, P., Li, M., Liu, X., Sarkhel, R., Tang, X., et al.: Shopping mmlu: A massive multi-task online shopping benchmark for large language models. arXiv preprint arXiv:2410.20745 (2024)

[170] Yang, J., Jimenez, C., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: Swe-agent: Agent-computer interfaces enable automated software engineering. Advances in Neural Information Processing Systems **37**, 50528–50652 (2024)

[171] Lu, C., Lu, C., Lange, R.T., Foerster, J., Clune, J., Ha, D.: The ai scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292 (2024)

[172] Gridach, M., Nanavati, J., Abidine, K.Z.E., Mendes, L., Mack, C.: Agentic ai for scientific discovery: A survey of progress, challenges, and future directions. arXiv preprint arXiv:2503.08979 (2025)

[173] Ifargan, T., Hafner, L., Kern, M., Alcalay, O., Kishony, R.: Autonomous llm-driven research—from data to human-verifiable research papers. NEJM AI **2**(1), 2400555 (2025)

[174] Liu, S., Lu, Y., Chen, S., Hu, X., Zhao, J., Lu, Y., Zhao, Y.: Drugagent: Automating ai-aided drug discovery programming through llm multi-agent collaboration. arXiv preprint arXiv:2411.15692 (2024)

[175] Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., Wang, Z., Wang, Z., Yin, F., Zhao, J., et al.: Exploring large language model based intelligent agents: Definitions, methods, and prospects. arXiv preprint arXiv:2401.03428 (2024)

[176] Schneider, J., Haag, S., Kruse, L.C.: Negotiating with llms: Prompt hacks, skill gaps, and reasoning deficits. arXiv preprint arXiv:2312.03720 (2023)

[177] Jang, J.Y., Shin, S., Gweon, G.: Minimal yet big impact: How ai agent back-channeling enhances conversational engagement through conversation persistence and context richness. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 14509–14521 (2024)

[178] Sudarshan, M., Shih, S., Yee, E., Yang, A., Zou, J., Chen, C., Zhou, Q., Chen, L., Singhal, C., Shih, G.: Agentic llm workflows for generating patient-friendly medical reports. arXiv preprint arXiv:2408.01112 (2024)

[179] Paul, A., Yu, C.L., Susanto, E.A., Lau, N.W.L., Meadows, G.I.: Agentpeertalk: Empowering students through agentic-ai-driven discernment of bullying and joking in peer interactions in schools. arXiv preprint arXiv:2408.01459 (2024)

[180] Wang, W., Ma, Z., Wang, Z., Wu, C., Chen, W., Li, X., Yuan, Y.: A survey of llm-based agents in medicine: How far are we from baymax? arXiv preprint arXiv:2502.11211 (2025)

[181] O'Grady, C.G., OG, C.: Agentic workflows in the practice of law—ai agents as ethics counsel. Arizona Legal Studies Discussion Paper, 25–03 (2024)

[182] Yu, Y., Yao, Z., Li, H., Deng, Z., Jiang, Y., Cao, Y., Chen, Z., Suchow, J., Cui, Z., Liu, R., *et al.*: Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. Advances in Neural Information Processing Systems **37**, 137010–137045 (2024)

[183] Xiao, Y., Sun, E., Luo, D., Wang, W.: Tradingagents: Multi-agents llm financial trading framework. arXiv preprint arXiv:2412.20138 (2024)

[184] Li, N., Gao, C., Li, M., Li, Y., Liao, Q.: Econagent: Large language model-empowered agents for simulating macroeconomic activities. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (2024)

[185] Williams, R., Hosseinichimeh, N., Majumdar, A., Ghaffarzadegan, N.: Epidemic modeling with generative agents. arXiv preprint arXiv:2307.04986 (2023)

[186] Zhang, J., Hou, Y., Xie, R., Sun, W., McAuley, J., Zhao, W.X., Lin, L., Wen, J.-R.: Agentcf: Collaborative learning with autonomous language agents for recommender systems. In: Proceedings of the ACM Web Conference 2024, pp. 3679–3689 (2024)

[187] Hua, W., Fan, L., Li, L., Mei, K., Ji, J., Ge, Y., Hemphill, L., Zhang, Y.: War and peace (waragent): Large language model-based multi-agent simulation of world wars. arXiv preprint arXiv:2311.17227 (2023)

[188] Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., Tu, Z.: Encouraging divergent thinking in large language models through multi-agent debate. arXiv preprint arXiv:2305.19118 (2023)

[189] Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., Liu, Z.: Chateval: Towards better llm-based evaluators through multi-agent debate. arXiv preprint arXiv:2308.07201 (2023)

[190] Piao, J., Lu, Z., Gao, C., Xu, F., Santos, F.P., Li, Y., Evans, J.: Emergence of human-like polarization among large language model agents. arXiv preprint arXiv:2501.05171 (2025)

[191] Ju, T., Wang, Y., Ma, X., Cheng, P., Zhao, H., Wang, Y., Liu, L., Xie, J., Zhang, Z., Liu, G.: Flooding spread of manipulated knowledge in llm-based multi-agent communities. arXiv preprint arXiv:2407.07791 (2024)

[192] Li, G., Hammoud, H., Itani, H., Khizbullin, D., Ghanem, B.: Camel: Communicative agents for" mind" exploration of large language model society. Advances in Neural Information Processing Systems **36**, 51991–52008 (2023)

[193] Hao, R., Hu, L., Qi, W., Wu, Q., Zhang, Y., Nie, L.: Chatllm network: More brains, more intelligence. AI Open (2025)

[194] Singh, M.P.: Consent as a foundation for responsible autonomy. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 12301–12306 (2022)

[195] Hu, S., Lu, C., Clune, J.: Automated design of agentic systems. arXiv preprint arXiv:2408.08435 (2024)

[196] Yuan, S., Song, K., Chen, J., Tan, X., Li, D., Yang, D.: Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. arXiv preprint arXiv:2406.14228 (2024)

[197] Argyle, L.P., Busby, E.C., Fulda, N., Gubler, J.R., Rytting, C., Wingate, D.: Out of one, many: Using language models to simulate human samples. Political Analysis **31**(3), 337–351 (2023)

[198] Wang, L., Zhang, J., Chen, X., Lin, Y., Song, R., Zhao, W.X., Wen, J.-R.: Recagent: A novel simulation paradigm for recommender systems. arXiv preprint arXiv:2306.02552 (2023)

[199] Moura, J.: Multi AI Agent Systems with crewAI. DeepLearning.AI Short Course. Lesson: Create agents to research and write an article (2024). https://www.deeplearning.ai/courses/multi-ai-agent-systems-with-crewai

[200] Wang, Z., Mao, S., Wu, W., Ge, T., Wei, F., Ji, H.: Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. arXiv preprint arXiv:2307.05300 (2023)

[201] Barenboim, L., Elkin, M., Pettie, S., Schneider, J.: The locality of distributed symmetry breaking. Journal of the ACM (JACM) **63**(3), 1–45 (2016)

[202] Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X.: Large language model based multi-agents: a survey of progress and challenges. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, pp. 8048–8057 (2024)

[203] Li, X., Wang, S., Zeng, S., Wu, Y., Yang, Y.: A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges. Vicinagearth **1**(1), 9 (2024)

[204] Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: International Conference on Machine Learning (2023)

[205] Surapaneni, R., Jha, M., Vakoc, M., Segal, T.: Announcing the Agent2Agent Protocol (A2A). Google Developers Blog. Accessed: 2026-10-17 (2025). https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[206] Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., Arik, S.: Chain of agents: Large language models collaborating on long-context tasks. Advances in Neural Information Processing Systems **37**, 132208–132237 (2024)

[207] Zhang, H., Du, W., Shan, J., Zhou, Q., Du, Y., Tenenbaum, J.B., Shu, T., Gan, C.: Building cooperative embodied agents modularly with large language models. arXiv preprint arXiv:2307.02485 (2023)

[208] Yamamoto, A., Ito, O., Katagiri, A., Koike, Y.: Dynamic knowledge integration in multi-agent systems for content inference. In: ICLR 2025 Workshop on Agentic AI for Science (2025)

[209] Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., Shmueli-Scheuer, M.: Survey on evaluation of llm-based agents. arXiv preprint arXiv:2503.16416 (2025)

[210] Wu, Y., Tang, X., Mitchell, T.M., Li, Y.: Smartplay: A benchmark for llms as intelligent agents. arXiv preprint arXiv:2310.01557 (2023)

[211] Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., et al.: Agentharm: A benchmark for measuring harmfulness of llm agents. arXiv preprint arXiv:2410.09024 (2024)

[212] ARC Prize Foundation: ARC-AGI-2: Abstraction and Reasoning Corpus for Artificial General Intelligence v2. GitHub. Accessed: 2025-04-16 (2025)

[213] Narajala, V.S., Narayan, O.: Securing agentic ai: A comprehensive threat model and mitigation framework for generative ai agents. arXiv preprint arXiv:2504.19956 (2025)

[214] Aher, G.V., Arriaga, R.I., Kalai, A.T.: Using large language models to simulate multiple humans and replicate human subject studies. In: International Conference on Machine Learning, pp. 337–371 (2023). PMLR

[215] Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., *et al.*: Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In: The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track (2024)

[216] Walke, H.R., Black, K., Zhao, T.Z., Vuong, Q., Zheng, C., Hansen-Estruch, P., He, A.W., Myers, V., Kim, M.J., Du, M., *et al.*: Bridgedata v2: A dataset for robot learning at scale. In: Conference on Robot Learning, pp. 1723–1736 (2023). PMLR

[217] Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., et al.: Measuring coding challenge competence with apps. arXiv preprint arXiv:2105.09938 (2021)

[218] Akata, E., Schulz, L., Coda-Forno, J., Oh, S.J., Bethge, M., Schulz, E.: Playing repeated games with large language models. arXiv preprint arXiv:2305.16867 (2023)

[219] Lin, J., Zhao, H., Zhang, A., Wu, Y., Ping, H., Chen, Q.: Agentsims: An open-source sandbox for large language model evaluation. arXiv preprint arXiv:2308.04026 (2023)

[220] Pan, Y., Kong, D., Zhou, S., Cui, C., Leng, Y., Jiang, B., Liu, H., Shang, Y., Zhou, S., Wu, T., Wu, Z.: Webcanvas: Benchmarking web agents in online environments. arXiv preprint arXiv:2406.12373 (2024)

[221] Turpin, M., Michael, J., Perez, E., Bowman, S.: Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. Advances in Neural Information Processing Systems **36**, 74952–74965 (2023)

[222] Longo, L., Brcic, M., Cabitza, F., Choi, J., Confalonieri, R., Del Ser, J., Guidotti, R., Hayashi, Y., Herrera, F., Holzinger, A., *et al.*: Explainable artificial intelligence (xai) 2.0: A manifesto of open challenges and interdisciplinary research directions. Information Fusion **106**, 102301 (2024)

[223] Qi, X., Huang, K., Panda, A., Henderson, P., Wang, M., Mittal, P.: Visual adversarial examples jailbreak aligned large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 21527–21536 (2024)

[224] Schroeder, D.T., Cha, M., Baronchelli, A., Bostrom, N., Christakis, N.A., Garcia, D., Goldenberg, A., Kyrychenko, Y., Leyton-Brown, K., Lutz, N., et al.: How malicious ai swarms can threaten democracy. arXiv preprint arXiv:2506.06299 (2025)

[225] Mok, A.: Data is fueling the ai revolution. what happens when it runs out? California Magazine (2025)

[226] Bengio, Y., Hu, E.J.: Scaling in the Service of Reasoning & Model-Based ML. https://yoshuabengio.org/2023/03/21/scaling-in-the-service-of-reasoning-model-based-ml/. Accessed: 2025-04-23 (2023)

[227] Ford, M.: Architects of Intelligence: The Truth About AI from the People Building It. Packt Publishing Ltd, Birmingham, UK (2018)

[228] Kurzweil, R.: The Singularity Is Near: When Humans Transcend Biology. Viking Penguin, New York (2005)

[229] Bostrom, N.: Superintelligence: Paths, Dangers, Strategies. Oxford University Press, Oxford, UK (2014)

[230] Wohlin, C.: Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, pp. 1–10 (2014)

[231] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., *et al.*: A survey on large language model based autonomous agents. Frontiers of Computer Science **18**(6), 186345 (2024)

[232] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., *et al.*: The rise and potential of large language model based agents: A survey. Science China Information Sciences **68**(2), 121101 (2025)

[233] Acharya, D.B., Kuppan, K., Divya, B.: Agentic ai: Autonomous intelligence for complex goals–a comprehensive survey. IEEE Access (2025)

[234] Plaat, A., Duijn, M., Stein, N., Preuss, M., Putten, P., Batenburg, K.J.: Agentic large language models, a survey. arXiv preprint arXiv:2503.23037 (2025)

[235] Manduchi, L., Pandey, K., Meister, C., Bamler, R., Cotterell, R., Däubener, S., Fellenz, S., Fischer, A., Gärtner, T., Kirchler, M., et al.: On the challenges and opportunities in generative ai. arXiv preprint arXiv:2403.00025 (2024)

[236] Sapkota, R., Roumeliotis, K.I., Karkee, M.: Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. Information Fusion, 103599 (2025)

[237] Khan, R.U., Aljuaid, H., Ning, Z.: A critical assessment of the rise of agentic ai, its capabilities, task domains and open challenges. Discover Artificial Intelligence **6**(1), 1343 (2026)